

Meshr-Lang

Documentation du langage

G. Prince - Architecte Principal

Version 0.4.0 - Septembre 2025

Contents

1 Meshr-Lang	6
1.1 Documentation du langage	6
2 Meshr-lang	7
2.1 Table des matiÃres	7
2.1.1 Introduction	7
2.1.2 Syntaxe et concepts de base	7
2.1.3 Organisation du code	7
2.1.4 MÃtadonnÃes et annotations	7
2.1.5 Types de donnÃes	8
2.1.6 MÃtriques et KPIs	8
2.1.7 ContrÃles avancÃs	8
2.1.8 Ressources et bonnes pratiques	8
2.1.9 Annexes	9
3 Introduction	9
3.1 Contexte et motivation	9
3.2 Philosophie du langage	9
3.3 Valeur ajoutÃe	10
3.4 Public cible	10
3.5 Vision Ã long terme	10
3.6 Objectif du langage	11
3.6.1 Vision et philosophie	11
3.6.2 Rationnel du langage	11
3.6.3 Objectifs principaux	12
3.6.4 Avantages clÃs	13
3.7 Syntaxe de base	13
3.7.1 Structure des fichiers	13
3.7.2 Style d'Ãclaratif	14
3.7.3 Mots-clÃs rÃservÃs	14
3.7.4 Commentaires	15
3.7.5 Types de valeurs primitives	16
3.7.6 Structure d'un module	17

3.8	D��claration des modules et imports	17
3.8.1	Concept et objectifs	17
3.8.2	Architecture modulaire	18
3.8.3	Syntaxe	18
3.8.4	S��mantique	18
3.8.5	Types d'imports support��s	19
3.8.6	Ordre des d��clarations	19
3.8.7	Export des d��clarations	20
3.8.8	�� Convention darborescence	22
3.9	Annotations	22
3.9.1	Syntaxe	22
3.9.2	R��gles	23
3.9.3	[2025-09-10] Annotations avec arguments typ��s	23
3.9.4	D��claration des types dannnotations	23
3.9.5	Types de base autoris��s pour les annotations	24
3.9.6	Contraintes sur les types String	25
3.9.7	Annotations Standard (StdLib)	28
3.10	��num��rations	31
3.10.1	Concept et objectifs	31
3.10.2	Types d'��num��rations	32
3.10.3	Syntaxe simple (inline)	32
3.10.4	Export group��	32
3.10.5	R��gles	32
3.10.6	Syntaxe de r��f��rence des valeurs enum	33
3.10.7	[2025-09-10] Enums avec attributs typ��s	33
3.10.8	Syntaxe enrichie	34
3.10.9	R��gles	34
3.10.10	Intervalles (Interval)	34
3.10.11	Types composites: List, Map, Range, Record	35
3.11	Records et Collections	39
3.11.1	Syntaxe des Records	39
3.11.2	Types de Collections	39
3.11.3	Collections imbriqu��es	40
3.11.4	Usage dans tous les contextes	41
3.11.5	R��gles s��mantiques	42
4	8	43
5	Types personnalis��s	43
5.1	Concept et objectifs	43
5.1.1	Principe de sp��cialisation	43
5.1.2	Avantages	43
5.2	Syntaxe de base	43
5.2.1	D��claration g��n��rale	43
5.2.2	Types String avec contraintes	43
5.2.3	Types num��riques avec contraintes	44
5.2.4	Types temporels avec contraintes	44
5.2.5	Autres types sp��cialis��s	44

5.3	Contraintes supportées	45
5.3.1	Contraintes String	45
5.3.2	Contraintes numériques	45
5.3.3	Contraintes temporelles	45
5.3.4	Contraintes spécialisées	45
5.4	Utilisation dans les déclarations	45
5.4.1	Dans les entités	45
5.4.2	Dans les records	46
5.4.3	Dans les relations	46
5.4.4	Dans les matrices	46
5.5	Types sealed	47
5.6	Règles sémantiques	47
5.6.1	Validation des contraintes	47
5.6.2	Résolution des types	47
5.6.3	Exemples d'erreurs	47
5.7	Bonnes pratiques	47
5.7.1	Nommage	47
5.7.2	Organisation	48
5.7.3	Contraintes	48
5.7.4	Exemple d'organisation	48
6	9	49
7	Entités et Relations	49
7.0.1	Entités	49
7.0.2	Types de Relations	50
7.0.3	Relations	51
7.0.4	Caractéristiques des Entités et Relations	52
7.1	Traits	52
7.1.1	Concept et rôle	52
7.1.2	Caractéristiques des traits	53
7.1.3	Différence avec les aspects	53
7.1.4	Syntaxe	53
7.1.5	Injection via <code>with</code>	54
7.1.6	Aspects dans les Traits	54
7.1.7	Règles de composition (sémantique)	55
7.1.8	[2025-09-09] Module = unité, namespace et fichier unique	56
7.2	Aspects	56
7.2.1	Concept et rôle	57
7.2.2	Caractéristiques des aspects	57
7.2.3	Syntaxe de base	57
7.2.4	Héritage et composition	58
7.2.5	Types d'Aspects	58
7.2.6	Champs et contraintes	59
7.2.7	Annotations sur les Aspects	59
7.2.8	Règles de composition	59
7.2.9	® Instanciation (future)	60

8	12	61
9	Politiques	61
9.1	Concept et objectifs	61
9.1.1	Principe de gouvernance d'Ãclarative	61
9.1.2	Domaines d'application	61
9.2	Syntaxe de base	61
9.2.1	Structure gÃnÃrale	61
9.3	Composants d'ÃtaillÃs	62
9.3.1	1. Verbatim (optionnel)	62
9.3.2	2. Scope (obligatoire)	62
9.3.3	3. Conditions	62
9.3.4	4. Pattern Matching	63
9.3.5	5. Actions	64
9.4	Exemples complets	65
9.4.1	Policy de gouvernance basique	65
9.4.2	Policy avec validation d'aspects multiples	65
9.4.3	Policy avec pattern matching avancÃ	66
9.4.4	Policy de qualitÃ des mÃtriques	66
9.5	RÃgles sÃmantiques	67
9.5.1	Validation des scopes	67
9.5.2	Conditions vs Pattern Matching	67
9.5.3	Actions et effets de bord	67
9.5.4	Patch operations	67
9.6	Bonnes pratiques	67
9.6.1	Organisation	68
9.6.2	Messages	68
9.6.3	Exemple d'organisation	68
10	13	69
11	Modificateur sealed	69
11.0.1	Syntaxe	69
11.0.2	RÃgles d'hÃritage et d'extension	69
11.0.3	Exemples valides	70
11.0.4	Exemples invalides	71
11.0.5	RÃgles sÃmantiques	71
12	MÃtriques (Metrics)	72
12.1	Introduction	72
12.1.1	Philosophie	72
12.2	Syntaxe de base	72
12.2.1	Structure minimale	72
12.2.2	Exemple simple	73
12.3	Composants d'ÃtaillÃs	73
12.3.1	1. Source de donnÃes	73
12.3.2	2. Calcul principal	73
12.3.3	3. UnitÃ de mesure	73

12.3.4	4. Outputs (optionnel)	74
12.3.5	5. Dimensions (optionnel)	74
12.3.6	6. Filtres (optionnel)	75
12.3.7	7. Agrégation avancée (optionnel)	75
12.3.8	8. Configuration temporelle (optionnel)	75
12.3.9	9. Aspects (optionnel)	75
12.4	Exemple complet	76
12.5	Règles sémantiques	77
12.5.1	Obligatoires	77
12.5.2	Optionnels	77
12.5.3	Validation	77
12.6	Artefacts définissables	78
12.6.1	1. domain	78
12.6.2	2. product	78
12.6.3	3. contract	78
12.6.4	4. aspect	78
12.6.5	5. team	79
12.6.6	6. policy	79
12.7	Décisions de conception	79
12.7.1	[2025-09-09] Syntaxe déclarative avec blocs { }	79
12.7.2	[2025-09-10] Visibilité par export explicite	79
12.7.3	[2025-09-10] Deux formes d'export : inline ou groupé	79
12.7.4	[2025-09-12] Modificateur sealed pour contrôler l'extensibilité	80
12.7.5	[2025-09-12] Types de collections typés et littéraux	80
12.7.6	[2025-09-12] Entités et Relations pour la modélisation de données	80
12.8	Bibliothèque Standard (StdLib)	81
12.8.1	Objectifs	81
12.8.2	Modules disponibles	81
12.8.3	Structure actuelle	87
12.8.4	Cas d'usage	88
12.8.5	Roadmap	89
12.9	À venir	89
12.10	Exemples pratiques et cas d'usage	90
12.10.1	Cas d'usage 1 : E-commerce et gestion des commandes	90
12.10.2	Cas d'usage 2 : Architecture Data Mesh - Domaine RH	95
12.10.3	Cas d'usage 3 : Secteur financier - Gestion des comptes	97
12.10.4	Cas d'usage 4 : Intégration technique - APIs et microservices	100
12.11	Bonnes pratiques et conventions	103
12.11.1	Conventions de nommage	103
12.11.2	Organisation des modules	104
12.11.3	Utilisation des traits et aspects	104
12.11.4	Documentation et commentaires	105
12.11.5	Gestion des erreurs et validation	105
12.11.6	Sécurité et conformité	106
12.11.7	Tests et validation	106
12.12	Annexes	106
12.12.1	Annexe A Grammaire EBNF (référence)	106
12.12.2	Annexe B Grammaire ANTLR (référence, Python target)	112

Figure 1: Logo Meshr-Lang

12.13	Annexe C : Guide complet des littÃraux	126
12.13.1	LittÃraux primitifs	126
12.13.2	LittÃraux composites	127
12.13.3	LittÃraux spÃcialisÃs	127
12.13.4	LittÃraux temporels	128
12.13.5	LittÃraux SQL	129
12.13.6	Exemples d'usage dans les annotations	130
12.13.7	Exemples d'usage dans les enums	131
12.13.8	Exemples d'usage dans les records	131
12.13.9	Exemples d'usage dans les aspects	132
12.13.10	Exemples d'usage dans les entitÃs	132
12.13.11	Conseils d'utilisation	133

1 Meshr-Lang

1.1 Documentation du langage

Version 0.4.0 - Septembre 2025

Avec support complet des politiques de gouvernance

G. Prince - Architecte Principal

2025-09-18

2 Meshr-lang

Documentation du langage

G. Prince - Architecte Principal

Version 0.4.0 - Septembre 2025

2.1 Table des matières

2.1.1 Introduction

- Objectif du langage
 - Vision et philosophie
 - Objectifs principaux
 - Avantages clés

2.1.2 Syntaxe et concepts de base

- Syntaxe de base
 - Structure des fichiers
 - Style déclaratif
 - Commentaires
 - Types de valeurs primitives
 - Structure d'un module

2.1.3 Organisation du code

- Déclaration des modules et imports
 - Concept et objectifs
 - Architecture modulaire
 - Syntaxe
 - Sémantique
 - Types d'imports supportés
 - Ordre des déclarations
 - Export des déclarations
 - □ Convention d'arborescence

2.1.4 Métadonnées et annotations

- Annotations
 - Syntaxe

- Règles
- Annotations avec arguments typés
- Déclaration des types d'annotations
- Types de base autorisés
- Contraintes sur les types String
- Annotations Standard (StdLib)

2.1.5 Types de données

- Énumérations
 - Concept et objectifs
 - Types d'Énumérations
 - Syntaxe simple (inline)
 - Export groupé
 - Règles
 - Enums avec attributs typés
- Records et Collections
- Entités et Relations
- Traits
- Aspects

2.1.6 Métriques et KPIs

- Métriques (Metrics)
 - Introduction
 - Syntaxe de base
 - Composants détaillés
 - Exemple complet
 - Règles sémantiques

2.1.7 Contrôles avancés

- Modificateur sealed
- Artefacts définissables

2.1.8 Ressources et bonnes pratiques

- Décisions de conception
- Bibliothèque Standard (StdLib)
- À venir
- Exemples pratiques et cas d'usage

- Bonnes pratiques et conventions

2.1.9 Annexes

- Annexes
 - Annexe A Grammaire EBNF (référence)
 - Annexe B Grammaire ANTLR (référence, Python target)
-

3 Introduction

Meshr-Lang est un langage de modélisation déclaratif conçu spécifiquement pour l'architecture **Data Mesh** et la gouvernance des données à l'échelle de l'entreprise. Il répond aux défis modernes de la gestion des données distribuées en offrant une syntaxe claire, expressive et alignée sur les principes de l'architecture Data Mesh.

3.1 Contexte et motivation

Dans un monde où les données deviennent le moteur de l'innovation, les organisations font face à des défis croissants :

- **Explosion du volume de données** : Multiplication des sources et formats
- **Complexité organisationnelle** : Données dispersées dans des silos métier
- **Exigences de conformité** : RGPD, SOX, HIPAA et autres réglementations
- **Besoin d'agilité** : Time-to-market critique pour les produits data
- **Qualité et fiabilité** : Confiance dans les données pour la prise de décision

L'architecture **Data Mesh** émerge comme une solution à ces défis, mais nécessite des outils et langages adaptés pour être mise en œuvre efficacement.

3.2 Philosophie du langage

Meshr-Lang s'inspire de plusieurs principes fondamentaux :

3.2.0.1 Déclaratif avant tout

- **Ce que vous voulez**, pas comment l'obtenir
- Focus sur la **sémantique métier** plutôt que l'implémentation technique
- **Lisibilité** et **expressivité** au cœur du design

3.2.0.2 Data Mesh Native

- **Domaine-centré** : Chaque module représente un domaine métier
- **Décentralisé** : Pas de dépendance à une architecture centralisée
- **Self-serve** : Outils et patterns intégrés pour l'autonomie des équipes

3.2.0.3 Pragmatique et Évolutif

- **Syntaxe simple** mais **puissante**
- **Extensibilité** via les annotations et aspects
- **Intégration** avec l'écosystème existant

3.3 Valeur ajoutée

Meshr-Lang apporte une **valeur unique** dans l'écosystème des langages de modélisation :

Aspect	Meshr-Lang	Alternatives
Focus Data Mesh	Natif	Générique
Gouvernance intégrée	Built-in	Externe
Syntaxe déclarative	Simple	Complexe
Annotations riches	Expressives	Basiques
Évolutivité	Modulaire	Monolithique

3.4 Public cible

Ce langage s'adresse à :

- **** Data Architects**** : Conception d'architectures Data Mesh
- **** Data Product Owners**** : Définition de produits de données
- **** Data Stewards**** : Gouvernance et qualité des données
- **» Data Engineers** : Implémentation et intégration
- **** Data Analysts**** : Compréhension et utilisation des données
- **** Compliance Officers**** : Conformité et audit

3.5 Vision à long terme

Meshr-Lang aspire à devenir **le standard de facto** pour la modélisation Data Mesh, en offrant :

- **** Adoption large**** dans l'industrie
- **** Écosystème riche**** d'outils et intégrations
- **** Communauté active**** de contributeurs
- **** Formation et certification**** pour les professionnels

3.6 Objectif du langage

Meshr-Lang est un langage déclaratif spécialisé conçu pour décrire et modéliser les artefacts d'une architecture **Data-as-a-Product** et d'entreprise. Il permet de définir de manière structurée et exécutable les domaines métier, produits de données, contrats, aspects (métadonnées), équipes, politiques, et leurs interrelations.

3.6.1 Vision et philosophie

Meshr-Lang s'inscrit dans la philosophie du **Data Mesh** et des architectures d'entreprise modernes ¹ :

- **La donnée est un produit** : Chaque dataset est traité comme un produit avec ses propres propriétés, contrats, et SLA
- **La documentation est vivante** : Le code source devient la source de vérité pour l'architecture
- **L'automatisation est centrale** : Les modèles déclaratifs génèrent automatiquement les artefacts techniques
- **La gouvernance est intégrée** : Les politiques et contraintes sont exprimées directement dans le modèle

3.6.2 Rationnel du langage

3.6.2.1 Pourquoi un nouveau langage ? L'écosystème actuel des langages de modélisation présente plusieurs limitations pour l'architecture Data Mesh :

**** Langages existants inadaptés : - UML **** : Trop générique, pas de support natif pour les concepts Data Mesh - **JSON Schema** : Syntaxe verbeuse, pas de support pour les relations complexes - **YAML** : Pas de validation de types, syntaxe peu expressive - **GraphQL Schema** : Limité aux APIs, pas de support pour la gouvernance - **OpenAPI** : Focus API uniquement, pas de modélisation métier

**** Besoins spécifiques Data Mesh : - Domaine-centré **** : Modélisation des domaines métier et leurs frontières - **Gouvernance intégrée** : Support natif pour les politiques et contraintes - **Relations complexes** : Modélisation des dépendances entre produits de données - **Métadonnées riches** : Annotations et aspects pour la documentation - **Évolutivité** : Support pour l'architecture distribuée et décentralisée

3.6.2.2 Choix de design 1. Syntaxe déclarative

```
// Ce que vous voulez , pas comment l'obtenir
entity Customer is
  id : String
```

```

    name : String
    email : String pattern "^[^@]+@[^@]+$"
end

```

2. Types riches et contraintes

```

// Types avec contraintes intÃ©grÃ©es
type Email is String pattern "^[^@]+@[^@]+$"
type Age is Integer range 0..150
type PhoneNumber is String pattern "^\+?[1-9]\d{1,14}$"

```

3. Annotations expressives

```

@Version("1.2.0")
@author(name="Data Team", email="data@company.com")
@Documented(summary="Customer data model")
@Confidentiality("internal")
entity Customer is
    // ...
end

```

4. Relations sÃ©mantiques

```

// Relations avec sÃ©mantique mÃ©tier
relation CustomerOrder from Customer(id) to Order(customerId)
relation ProductCategory from Product(id) to Category(id)

```

3.6.2.3 Architecture du langage Composants clÃ©s :

1. **** Modules**** : UnitÃ©s de dÃ©ploiement et de versioning
2. **** Types**** : SystÃ©me de types riche avec contraintes
3. **** Relations**** : ModÃ©lisation des dÃ©pendances mÃ©tier
4. **** Annotations**** : MÃ©tadonnÃ©es et documentation intÃ©grÃ©e
5. **** Aspects**** : Comportements transversaux (sÃ©curitÃ©, qualitÃ©, etc.)

Principe de composition : - **Modulaire** : Chaque module est autonome - **Composable** : Les modules peuvent s'assembler - **Extensible** : Nouvelles fonctionnalitÃ©s via les aspects - **Validable** : VÃ©rification statique des contraintes

3.6.3 Objectifs principaux

Meshr-Lang vise Ã© offrir une syntaxe lisible, formelle et exÃ©cutable pour :

3.6.3.1 Documentation vivante

- **Structurer la documentation** : Transformer la documentation statique en modèles exécutables
- **Maintenir la cohérence** : Assurer que la documentation reste synchronisée avec l'implémentation
- **Faciliter la compréhension** : Rendre l'architecture d'entreprise accessible à tous les acteurs

3.6.3.2 Génération automatique

- **Représentations techniques** : Générer automatiquement YAML, Rego, Terraform, OpenAPI, etc.
- **Code boilerplate** : Réduire la duplication et les erreurs de codage manuel
- **Validation continue** : Détecter les incohérences et violations de politiques

3.6.3.3 Gouvernance intégrée

- **Registres et catalogues** : Alimenter automatiquement les plateformes de gouvernance
- **Self-service** : Permettre aux équipes de découvrir et consommer les données facilement
- **Conformité** : Intégrer les exigences réglementaires (RGPD, SOX, etc.) dans le modèle

3.6.4 Avantages clés

- **Lisibilité** : Syntaxe claire et expressive inspirée des meilleures pratiques
 - **Extensibilité** : Architecture modulaire permettant l'ajout de nouveaux concepts
 - **Intégration** : Compatible avec l'écosystème moderne (CI/CD, IaC, observabilité)
 - **Validation** : Vérification statique des modèles et détection d'erreurs précoces
-

3.7 Syntaxe de base

Meshr-lang est un langage de modélisation déclaratif conçu pour décrire des architectures d'entreprise, des produits, des domaines métier et leurs relations. Sa syntaxe s'inspire de langages modernes comme HCL (HashiCorp Configuration Language), Kotlin DSL, et GraphQL Schema Definition Language (SDL).

3.7.1 Structure des fichiers

Extension de fichier : .meshr

Tous les fichiers Meshr-lang utilisent l'extension `.meshr` et suivent une structure modulaire où¹ chaque fichier représente un module autonome contenant des déclarations de types, d'entités, d'annotations, et d'autres artefacts métier.

3.7.2 Style déclaratif

Meshr-lang adopte un style **déclaratif à blocs** qui privilégie la lisibilité et la structure hiérarchique :

- **Blocs imbriqués** : Les déclarations sont organisées en blocs logiques avec indentation
- **Syntaxe claire** : Utilisation de mots-clés explicites et de ponctuation minimale
- **Lisibilité** : Structure qui reflète naturellement la hiérarchie des concepts métier

3.7.3 Mots-clés réservés

Les mots suivants sont **réservés** par le langage et **ne peuvent pas être utilisés** comme identifiants de champs, variables ou noms d'entités :

3.7.3.1 Déclarations principales

- `module`, `import`, `export`
- `entity`, `enum`, `trait`, `aspect`, `annotation`, `record`, `relation`
- `type`, `sealed`, `bidirectional`

3.7.3.2 Métriques (nouveau v0.2.0)

- `metric`, `source`, `calculation`, `aggregation`, `unit`
- `outputs`, `dimensions`, `filters`, `temporal`
- `window`, `refresh_frequency`, `historical_depth`
- `match`, `or` (pattern matching)

3.7.3.3 Relations et composition

- `from`, `to`, `with`, `extends`, `implements`
- `of`, `is`, `end`, `aspects`

3.7.3.4 Annotations et modificateurs

- `required`, `optional`, `abstract`

```
// INCORRECT - 'dimensions' est un mot-clé réservé
entity Product is
    dimensions : String // Erreur de compilation !
```

```
end
```

```
// CORRECT - Utiliser un nom différent
```

```
entity Product is
```

```
    product_dimensions : String // OK
```

```
end
```

```
// INCORRECT - 'source' est réservé pour les matrices
```

```
relation DataFlow is
```

```
    from Source(id) to Target(id)
```

```
    source : String // Erreur de compilation !
```

```
end
```

```
// CORRECT
```

```
relation DataFlow is
```

```
    from Source(id) to Target(id)
```

```
    data_source : String // OK
```

```
end
```

3.7.4 Commentaires

Les commentaires permettent d'ajouter des explications et de la documentation directement dans le code source :

- **Commentaires de ligne** : Commencent par // et s'étendent jusqu'à la fin de la ligne
- **Commentaires de bloc** : Délimités par /* et */, peuvent s'étendre sur plusieurs lignes
- **Ignorés par le parser** : Les commentaires ne font pas partie de l'AST (Abstract Syntax Tree)

```
// Ceci est un commentaire de ligne
```

```
entity Customer {
```

```
    // Commentaire sur un champ
```

```
    name: String
```

```
}
```

```
/*
```

```
 * Ceci est un commentaire de bloc
```

- * qui peut s'attendre sur plusieurs lignes
- * pour documenter des concepts complexes
- */

3.7.5 Types de valeurs primitives

Meshr-lang supporte plusieurs types de valeurs primitives pour exprimer des données de base :

3.7.5.1 Chaînes de caractères

- **Syntaxe** : "texte entre guillemets doubles"
- **Usage** : Noms, descriptions, identifiants, valeurs textuelles
- **Exemples** : "Customer", "John Doe", "production"

3.7.5.2 Valeurs booléennes

- **Syntaxe** : true ou false
- **Usage** : Flags, conditions, propriétés binaires
- **Exemples** : isActive : true, isPublic : false

3.7.5.3 Nombres

- **Nombres décimaux** : 42, 3.14, -15.5
- **Nombres hexadécimaux** : 0xFF, 0x00FF00, 0x1A2B3C
- **Usage** : Compteurs, identifiants numériques, valeurs de configuration

3.7.5.4 Noms qualifiés

- **Syntaxe** : module.Type.EnumValue
- **Usage** : Références des types, annotations, ou constantes d'autres modules
- **Exemples** : privacy.Level.HIGH, core.Status.ACTIVE

3.7.5.5 Intervalles temporels

- **Syntaxe** : Interval <valeur> <unité>
- **Usage** : Durées, périodes, délais d'expiration
- **Exemples** :
 - Interval 12 month : 12 mois
 - Interval -1 day : -1 jour (hier)
 - Interval "2024-01" year to month : de janvier 2024 à maintenant

3.7.6 Structure d'un module

Un module Meshr-lang suit une structure hiérarchique bien définie :

1. **Déclarations d'import** (optionnelles)
2. **Déclaration du module** (obligatoire)
3. **Déclarations d'export** (optionnelles)
4. **Déclarations d'artefacts** (entités, énumérations, types, etc.)

```
// Ceci est un commentaire simple
```

```
/*  
Ceci est un commentaire  
sur plusieurs lignes  
*/
```

3.8 Déclaration des modules et imports

Le **module** est l'unité fondamentale d'organisation dans Meshr-lang. Il représente un espace de noms versionné et autonome qui contient des déclarations d'artefacts métier (entités, énumérations, types, etc.). Chaque fichier .meshr correspond à un module unique.

3.8.1 Concept et objectifs

Un module Meshr-lang sert à :

- **Encapsuler** : Regrouper des concepts métier liés dans un espace de noms cohérent
- **Versionner** : Permettre l'évolution des API et la gestion des dépendances
- **Réutiliser** : Faciliter l'import et la réutilisation de composants entre modules
- **Organiser** : Structurer l'architecture d'entreprise en composants modulaires
- **Isoler** : Définir des frontières claires entre différents domaines métier

3.8.2 Architecture modulaire

Meshr-lang encourage une architecture modulaire o  1 :

- **Un module = un domaine m  tier** : Chaque module repr  sente un domaine d'expertise sp  cifique
- **D  pendances explicites** : Les relations entre modules sont d  clar  es via des imports
- **Namespace hi  rarchique** : Les noms de modules suivent une convention hi  rarchique (ex: core.types, marketing.analytics)
- **Versioning s  mantique** : Chaque module peut   tre versionn   ind  pendamment

3.8.3 Syntaxe

```
@experimental
@version("0.2.0")
module marketing.analytics
```

```
// Import simple
import Customer from marketing.shared
```

```
// Import group   (recommand  )
import { DataClassification , WithSecurity } from meshr.security
import { DataStewardship , WithStewardship } from meshr.governance
import { Version , Author , Documented } from meshr.annotations
```

```
// Import wildcard
import * from meshr.types
```

```
// autres d  clarations
```

3.8.4 S  mantique

- **Un fichier .meshr = un module unique.**
- Le module d  finit le **namespace** des artefacts qu'il contient.
- Les import permettent d'acc  der    des symboles publics d'autres modules.
- Des **annotations peuvent pr  c  der** la d  claration module (ex.: @experimental, @version("...")).

3.8.5 Types d'imports supportés

```
import Customer from marketing.shared
```

Importe un seul Élément (enum, aspect, trait, etc.) d'un module.

```
import { DataClassification , WithSecurity } from meshr.security
```

Importe plusieurs Éléments spécifiques d'un module. C'est la méthode recommandée car elle est explicite et Évite les conflits de noms.

```
import * from meshr.types
```

Importe tous les Éléments exportés d'un module. Utilisez avec précaution pour Éviter les conflits de noms.

3.8.6 Ordre des déclarations

L'ordre correct dans un module Meshr-Lang est :

```
module mon.module
```

```
// 1. Imports (après la déclaration du module)
```

```
import { Item1 , Item2 } from autre.module
```

```
// 2. Exports (après les imports)
```

```
export { MonItem }
```

```
// 3. Déclarations (enums, aspects , traits , entités , etc.)
```

```
enum MonEnum is ("value1", "value2")
```

```
end
```

- Les import permettent d'accéder à des symboles **explicitement exportés** d'autres modules.

- Les éléments d'un module **ne sont pas visibles depuis l'extérieur** s'ils ne sont pas précédés du mot-clé `export`.
- L'import de `{ * }` signifie **importer tous les symboles exportés** du module cible.
- L'import de `{ A, B }` permet une **importation sélective**.
- Les accolades `{ }` sont **optionnelles uniquement si un seul identifiant** est importé.
- Si plusieurs identifiants sont importés d'un même module, l'usage des accolades est **obligatoire**.

3.8.7 Export des déclarations

Deux formes d'export sont possibles dans un module :

```
export enum SensitivityLevel is (public , internal , restricted )
```

- La déclaration est rendue publique immédiatement.
- Facile à lire mais pénalisant si plusieurs artefacts sont à exporter.

```
export { PIICategory , SensitivityLevel }
```

```
enum PIICategory is (contact , identity , financial , health , biometric , location )
enum SensitivityLevel is (public , internal , restricted , confidential )
```

- Peut apparaître **n'importe où dans le module** pour une meilleure organisation.
- Permet de déclarer tous les artefacts exportés en un seul point.
- Aucune déclaration listée dans ce bloc n'a besoin du mot-clé `export` en ligne.
- **Flexibilité maximale** : plusieurs exports groupés autorisés pour organiser le code par sections logiques.

```
@Version("1.0.0")
module my.data
```

```
import { DataClassification } from meshr.security
```

```
// Export des types de base
export { UserStatus, Department }

enum UserStatus is ("active", "inactive", "pending")
enum Department is ("hr", "finance", "engineering")

// Export des entités principales
export { User, Profile }

entity User is
  id : String
  name : String
  status : UserStatus
end

entity Profile is
  userId : String
  department : Department
end

// Export des relations
export { UserProfile }

relation UserProfile from User(id) to Profile(userId)
```

3.8.7.4 Règles de priorité d'export

1. Une déclaration précédée de export est **immédiatement publique**.
2. Une déclaration listée dans export { ... } devient **publique à posteriori**.
3. Un même nom peut apparaître dans les deux formes (toléré).
4. Toute déclaration **non exportée explicitement** est **privée** au module.
 - **Un fichier .meshr = un module unique.**
 - Le module définit le **namespace** des artefacts qu'il contient.
 - Les import permettent d'accéder à des symboles publics d'autres modules.
 - Les annotations @module_* sont optionnelles mais encouragées pour documenter les modules.
 - Les import permettent d'accéder à des symboles **explicitement exportés** d'autres modules.

- Les éléments d'un module **ne sont pas visibles depuis l'extérieur** s'ils ne sont pas précédés du mot-clé `export`.
- `import { * }` signifie **importer tous les symboles exportés** du module cible.
- `import { A, B }` permet une **importation sélective**.
- Les accolades `{ }` sont **optionnelles uniquement si un seul identifiant** est importé.
- Si plusieurs identifiants sont importés d'un même module, l'usage des accolades est **obligatoire**.

3.8.8 □ Convention d'arborescence

Par convention (non obligatoire), l'arborescence des fichiers reflète les noms qualifiés des modules :

Module	Fichier
<code>core.types</code>	<code>modules/core/types.meshr</code>
<code>marketing.analytics</code>	<code>modules/marketing/analytics.meshr</code>

3.9 Annotations

Les **annotations** permettent d'ajouter des métadonnées structurées sur n'importe quelle déclaration du langage Meshr (module, product, domain, enum, etc.). Elles sont inspirées de langages comme Java, Kotlin ou GraphQL.

3.9.1 Syntaxe

- Une annotation commence par `@` suivie de son nom.
- Elle peut recevoir :
 - Aucun argument : `@experimental`
 - Une seule valeur : `@since("1.2.0")`
 - Une ou plusieurs paires clé-valeur : `@author(name="Greg")`
 - Des arguments mixtes : `@scope(level=ScopeLevel.Internal)`
- Plusieurs annotations peuvent être empilées au-dessus d'une déclaration.

```
@experimental
@version("0.2.0")
@scope(level=ScopeLevel.Internal)
@author(
  name="Greg",
  email="greg@example.com"
```

```
)
@deprecated(reason="Use new module instead")
@confidentiality ( privacy . Level . HIGH )
module metadata . annotations
```

3.9.2 Règles

- Les valeurs peuvent être :
 - des chaînes : "texte"
 - des identifiants : Internal
 - des chemins qualifiés : privacy.Level.HIGH
- Les paires clé=valeur peuvent être combinées dans une annotation.
- Les annotations sont **optionnelles** et **non normatives** mais peuvent être exploitées par les outils CLI ou LSP.

3.9.3 [2025-09-10] Annotations avec arguments typés

- **Contexte** : Besoin d'enrichir les artefacts avec des métadonnées structurées.
- **Décision** : Support des annotations avec arguments (valeur unique ou paires clé/valeur), et valeurs typées (string , boolean, number y compris hex , qualifiedName, interval).
- **Pourquoi** : Permet une expressivité maximale tout en restant compatible avec une grammaire ANTLR claire et extensible.

3.9.4 Déclaration des types d'annotations

Meshr permet de déclarer des **types d'annotations** personnalisés à l'aide du mot-clé annotation.

```
@Target( annotation )
@Retention( model )
annotation Documented is
    required summary : String
    optional description : String = ""
    optional deprecated : Boolean = false
```

end

3.9.4.2 Sémantique

- Les types d'annotations sont eux-mêmes annotables (@Target, @Retention, @Repeatable).
- Le bloc interne suit la forme :
 - required : la valeur doit obligatoirement être fournie à l'usage, **aucune valeur par défaut autorisée**.
 - optional : la valeur est optionnelle, et peut être associée à une valeur par défaut.
- Si une annotation typée est utilisée sans renseigner un champ optionnel, alors la valeur par défaut s'applique.
- Les types autorisés incluent les **types de base** (String, Boolean, Integer, Float, Double, Date, Datetime, Time, Timestamp, Geography, Bytes, Json, Sql, Interval, Range) et les **noms qualifiés** (enums, types importés).
- Le mot-clé end est requis pour clore la déclaration.

3.9.4.3 Règle sémantique required vs optional

- required : champ **obligatoire, sans valeur par défaut**, doit être explicitement renseigné à l'usage.
- optional : champ **optionnel, avec ou sans valeur par défaut**.
 - Si valeur par défaut spécifiée □ utilisée si champ non renseigné.
 - Sinon □ champ absent à l'usage.

3.9.5 Types de base autorisés pour les annotations

Les types suivants peuvent être utilisés dans les déclarations d'annotations :

- Boolean : valeurs true ou false
- String : chaînes de caractères délimitées par des guillemets
- Integer : nombre entier (ex: 42)
- Float : nombre décimal (ex: 3.14)
- Double : précision étendue (équivalent sémantique à Float pour l'instant)
- Date, Datetime, Time, Timestamp : types temporels
- Geography : localisation géographique (future extension)
- Bytes : données binaires
- Json : valeurs encodées en JSON (future extension)
- Sql : requêtes SQL (ex: Sql"SELECT * FROM users")
- Interval : intervalle sur un type temporel (ex: Interval 2 hour)
- Range : intervalle contigu entre deux valeurs ordonnées (ex: Range 1..10)

3.9.6 Contraintes sur les types String

Les types String peuvent être enrichis avec des contraintes pour valider le format et la longueur des chaînes :

```
// Contrainte de pattern (regex)
email : String pattern "^[^@]+@[^@]+$"

// Contrainte de longueur
username : String length 3..20

// Contraintes combinées
phone : String pattern "[0-9]{10,15}$" length 10..15
```

3.9.6.2 Types de contraintes

- **pattern** : expression régulière pour valider le format
 - Exemple email : String pattern "^[^@]+@[^@]+\$"
 - Exemple téléphone : String pattern "[0-9]{10,15}\$"
 - Exemple URL : String pattern "^https ?://.*"
- **length** : bornes sur la longueur de la chaîne
 - Longueur fixe : String length 10..10
 - Longueur variable : String length 1..100
 - Longueur minimale : String length 5..

3.9.6.3 Utilisation dans tous les contextes Les contraintes String peuvent être utilisées partout où un type String est autorisé :

```
// Dans les annotations
annotation UserProfile is
    required email : String pattern "^[^@]+@[^@]+$"
    required username : String length 3..20
    optional phone : String pattern "[0-9]{10,15}$" length 10..15
end

// Dans les enums
enum HttpStatus(code: Integer, message: String pattern "[A-Z][a-z ]+$") is (
    ok(code = 200, message = "OK"),
    not_found(code = 404, message = "Not Found")
)
```

```
// Dans les records
record Contact is
  name : String length 1..100
  email : String pattern "^[^@]+@[^@]+$"
  phone : String pattern "[0-9]{10,15}$"
end

// Dans les traits
trait Identifiable is
  id : String pattern "[a-zA-Z0-9_-]+$" length 1..50
  name : String length 1..100
end
```

3.9.6.4 Règles sémantiques

- Les contraintes pattern et length peuvent être combinées
- L'ordre des contraintes n'est pas significatif
- Les contraintes sont optionnelles : String sans contrainte est valide
- Les erreurs de contraintes (double pattern, length incompatible) sont détectées par l'analyseur sémantique

Les annotations peuvent utiliser les valeurs suivantes:

- Chaînes: "abc"
- Booléens: true, false
- Numériques: 42, 3.14, 0xFF, 0x00FF00
- Noms qualifiés: privacy.Level.HIGH
- Intervalles: voir section d'ici ci-dessous

Les annotations peuvent également être utilisées comme type d'attribut.

```
annotation Repeatable is
  optional value : Boolean = true
end
```

```
enum Color(value: Integer) is (
  red(value = 0xFF0000),
  green(value = 0x00FF00),
  blue(value = 0x0000FF)
)
```

```
// Type d'annotation avec plusieurs champs
@Target(annotation)
@Retention(model)
annotation Example is
    required name : String
    optional version : String = "1.0"
    optional experimental : Boolean = false
end
```

```
// Annotation simple booléenne
@Target(annotation)
@Retention(model)
annotation Repeatable is
    optional value : Boolean = true
end
```

```
@Example(name="MeshrLang")
annotation Test1 is end
```

```
@Example(name="MeshrLang", experimental=true)
annotation Test2 is end
```

3.9.6.7 Tests invalides Les tests invalides sont implémentés dans le fichier `invalid-annotation-decl-test.mesh` et couvrent :

- **Erreurs de syntaxe dans les annotations** : Arguments d'annotations mal formés
- **Champs required non renseignés** : Validation des champs obligatoires
- **Redondance entre champs** : Détection des déclarations en double
- **Mauvais types** : Validation des types dans les annotations
- **Syntaxe invalide** : Structures d'annotations mal formées

Exemple d'erreur détectée :

```
@Target(annotation) // Erreur : argument mal formé
@Retention(model)
annotation InvalidAnnotation is
    optional : String = "missingName"
end
```

3.9.7 Annotations Standard (StdLib)

La bibliothèque standard Meshr-Lang fournit un ensemble d'annotations pré-définies dans le module `meshr.annotations`, incluant les méta-annotations et les annotations communes.

```
module mon.module

// Import des annotations standard
import {
  Target, Retention, Repeatable,
  Experimental, Deprecated, Version, Author,
  Scope, Confidentiality, Documented
} from meshr.annotations

export { MonEntite }
```

3.9.7.2 Méta-annotations

3.9.7.2.1 @Target Spécifie sur quels types d'éléments l'annotation peut être utilisée.

```
@Target(" all ")           // Tous les éléments
@Target(" annotation ")     // Seulement les annotations
@Target(" entity ")         // Seulement les entités
@Target(" module ")         // Seulement les modules
```

Valeurs possibles : - " all " : Tous les éléments - " annotation " : Annotations - " enum " : Énumérations - " module " : Modules - " record " : Records - " trait " : Traits - " aspect " : Aspects - " entity " : Entités - " type_relation " : Types de relations - " relation " : Relations

3.9.7.2.2 @Retention Spécifie la durée de vie de l'annotation.

```
@Retention(compile) // Supprime à la compilation
@Retention(model)    // Conserve dans le modèle
@Retention(export)   // Exporte avec l'artefact
```

3.9.7.2.3 @Repeatable Indique si l'annotation peut être utilisée plusieurs fois sur le même élément.

```
@Repeatable(true) // Peut être répété
@Repeatable(false) // Ne peut pas être répété
```

3.9.7.3 Annotations communes

3.9.7.3.1 @Experimental Marque un ÃlÃment comme expÃrimental.

@Experimental(reason="Feature is experimental and may change")

3.9.7.3.2 @Deprecated Marque un ÃlÃment comme dÃprÃciÃ.

```
@Deprecated(  
    reason="Use new OrderV2 entity instead",  
    since="2024-01-01",  
    replacement="OrderV2"  
)
```

3.9.7.3.3 @Version SpÃcifie la version d'un ÃlÃment.

@Version("1.2.0")

3.9.7.3.4 @Author SpÃcifie l'auteur d'un ÃlÃment.

```
@Author(  
    name="Data Team",  
    email="data@company.com",  
    organization="ACME Corp"  
)
```

3.9.7.3.5 @Scope SpÃcifie la portÃe d'un ÃlÃment.

@Scope(level="internal", description="Internal use only")

Niveaux possibles : - "public" : Public - "internal" : Interne - "private" : PrivÃ

3.9.7.3.6 @Confidentiality SpÃcifie le niveau de confidentialitÃ.

```
@Confidentiality(  
    level="confidential",  
    classification="PII",  
    handling_instructions="Encrypt at rest"  
)
```

Niveaux possibles : - "public" : Public - "internal" : Interne - "confidential" : Confidentiel -
"restricted" : Restreint

3.9.7.3.7 @Documented Ajoute de la documentation structurée.

```
@Documented(  
    summary="Customer entity with personal information",  
    description="Core entity for customer data management",  
    examples="See examples/ directory",  
    see_also="CustomerV2 entity"  
)
```

3.9.7.4 Exemples d'utilisation

```
@Version("1.0.0")  
@Author(name="Data Team", email="data@company.com")  
@Scope(level="internal")  
@Confidentiality(level="confidential")  
@Documented(summary="Customer management module")  
module examples.customer
```

```
@Version("1.2.0")  
@Author(name="Product Team")  
@Documented(summary="Customer entity with personal information")  
@Experimental(reason="New customer model, API may change")  
entity Customer is  
    id : String  
    name : String  
end
```

```
@Target("all")  
@Retention(model)  
@Repeatable(true)  
annotation QualityGate is  
    required level : String  
    optional automated : Boolean = true  
    optional reviewer : String = ""  
end
```

```
@QualityGate(level="high", reviewer="senior-dev")
```

```
entity CriticalEntity is
  id : String
end
```

3.9.7.5 Intégration avec la stdlib Les annotations s'intègrent parfaitement avec les autres modules de la stdlib :

```
import { DataClassification , WithSecurity } from meshr.security
import { Version , Author , Documented } from meshr.annotations

@Version("1.0.0")
@author(name="Security Team")
@Documented(summary="Secure customer entity")
entity SecureCustomer with WithSecurity is
  id : String

  aspects {
    DataClassification {
      level: "confidential",
      encryption_required: true
    }
  }
end
```

3.10 Énumérations

Les **Énumérations** (ou enums) sont des types de données qui permettent de définir un ensemble fini et ordonné de valeurs symboliques. Elles sont particulièrement utiles pour représenter des catégories, des statuts, des niveaux de priorité, ou tout autre concept métier qui peut être exprimé par un nombre limité d'options prédéfinies.

3.10.1 Concept et objectifs

Les Énumérations servent à :

- **Limiter les choix** : Restreindre les valeurs possibles à un ensemble prédéfini et valide
- **Améliorer la lisibilité** : Remplacer des valeurs numériques ou textuelles par des noms explicites

- **Assurer la cohérence** : Garantir que seules des valeurs valides sont utilisées dans le système
- **Faciliter la maintenance** : Centraliser la définition des valeurs possibles
- **Documenter le domaine** : Exprimer explicitement les options disponibles dans un contexte métier

3.10.2 Types d'Enums

Meshr-lang supporte deux formes d'Enums :

3.10.2.1 Enums simples

- **Valeurs symboliques** : Liste de noms représentant les options possibles
- **Usage** : Statuts, catégories, niveaux de priorité
- **Exemple** : (active, inactive, pending)

3.10.2.2 Enums avec paramètres

- **Valeurs enrichies** : Chaque valeur peut avoir des propriétés associées
- **Usage** : Codes d'erreur avec messages, statuts avec attributs
- **Exemple** : active(code = 200, message = "OK")

3.10.3 Syntaxe simple (inline)

```
export enum PIICategory is (contact, identity, financial, health, biometric, location)
```

- Le mot-clé `enum` introduit une Enum nommée.
- L'utilisation de `export` rend l'Enum visible à l'extérieur du module.
- Les valeurs sont listées entre parenthèses, séparées par des virgules.
- Cette forme convient aux déclarations rapides et lisibles.

3.10.4 Export groupé

```
export { PIICategory }
```

```
enum PIICategory is (contact, identity, financial, health, biometric, location)
```

- L'Enum est déclaré sans export, puis rendue publique via un bloc `export { ... }`.
- Cette forme permet de centraliser tous les artefacts publics après les imports.

3.10.5 Règles

- Les valeurs d'Enum peuvent être des **identifiants** ou des **chaînes** (ex: "export").

- Les identifiants de valeurs doivent être uniques et respectent la casse.
- Une ÉnumÉration peut être annotée, comme tout autre artefact.
- Le nom de l'ÉnumÉration doit être un identifiant valide.

3.10.6 Syntaxe de rÉfÉrence des valeurs enum

Les valeurs d'ÉnumÉration sont rÉfÉrencÉes avec la **notation pointÉe** EnumName.value :

```
// DÉclaration avec syntaxe mixte (identifiants + chaÉ@nes)
enum ApplicationStatus is (
    submitted,                // Identifiant simple
    "under review",           // ChaÉ@ne avec espace
    "screening passed",       // Espace au lieu d'underscore
    interviewing,             // Identifiant simple
    hired,                    // Identifiant simple
    "offer extended"          // ChaÉ@ne avec espace
)

// RÉfÉrence uniforme avec notation pointÉe
filters {
    status == ApplicationStatus.submitted,        // RÉfÉrence identifiant
    status == ApplicationStatus."under review",    // RÉfÉrence chaÉ@ne
    status == ApplicationStatus.hired              // RÉfÉrence identifiant
}
```

3.10.6.2 Avantages de cette approche

- **LisibilitÉ mÉtier** : Valeurs naturelles ("under review" vs under_review)
- **FlexibilitÉ** : MÉlange identifiants/chaÉ@nes selon les besoins
- **CohÉrence** : Syntaxe de rÉfÉrence uniforme avec .
- **Évolution** : Ajout facile de nouvelles valeurs complexes

3.10.7 [2025-09-10] Enums avec attributs typÉs

- **Contexte** : Il est utile de donner aux ÉnumÉrations des attributs associÉs É chaque valeur (ex. code couleur, libellÉ, gravitÉ).
- **DÉcision** : On introduit une forme enrichie de dÉclaration denum, oÉ1 chaque valeur peut porter une ou plusieurs **propriÉtÉs typÉes**, similaires aux enum class de Kotlin.

- **Pourquoi** : Cela augmente l'expressivité du langage, permet des relations croisées entre enums, et reste compatible avec les aspects et politiques du langage.

3.10.8 Syntaxe enrichie

```
enum Color(value: Integer) is (
    red(value = 0xFF0000),
    green(value = 0x00FF00),
    blue(value = 0x0000FF)
)
```

```
enum Severity(color: Color, severity: String = "none") is (
    none(color = Color.blue),
    medium(color = Color.green, severity = "medium"),
    high(color = Color.red, severity = "high")
)
```

Remarque : les valeurs entières peuvent être exprimées en hexadécimal (ex: 0xFF0000) pour représenter des couleurs ou des flags binaires.

3.10.9 Règles

- Une énumération peut définir une **signature d'attributs** dans (...) immédiatement après le nom.
- Chaque valeur de l'énumération doit fournir des arguments nommés (ex : color = Color.red).
- Les attributs peuvent avoir une **valeur par défaut**, comme severity: String = "none".
- Les types d'attributs peuvent être des **types de base** (cf. liste) ou des **noms qualifiés** (ex.: autre enum).
- L'ordre des arguments dans les valeurs n'a pas besoin de suivre celui de la signature.

¹ Les nombres peuvent être fournis en **hexadécimal** (0xFF0000) pour des codes couleur, et les **intervalles** sont supportés dans les valeurs d'annotation.

3.10.10 Intervalles (Interval)

Les littéraux d'intervalle sont supportés dans les annotations via la forme Interval

Formes supportées:

- Partie simple: Interval 12 month, Interval -1 day, Interval 2 hour
- Avec bornes textuelles + précision: Interval "2024-01" year to month
- Précisions supportées: year, quarter, month, week, day, hour, minute, second, millisecond, microsecond

Notes:

- Le signe – est autorisé devant les nombres d'intervalle: Interval -10 day.
- Les nombres hexadécimaux sont acceptés: Interval 0x10 second.

3.10.11 Types composites: List, Map, Range, Record

Meshr supporte des types composites utilisables dans les signatures d'annotations et les déclarations d'annotations via typeRef:

- List of T
- Map of K to V
- Range of T
- record référencé par nom (voir ci-dessous)

Exemples de types:

```
annotation Meta is
  required owner : String
  optional tags : List of String
  optional conf : Map of String to Integer
  optional span : Range of Integer
end
```

```
record Address is
  street : String
  zipcode : Integer
  extras : Map of String to Integer
end
```

3.10.11.1 Valeurs par défaut dans record Les champs d'un record peuvent recevoir une valeur par défaut:

```
record Address is
  street : String = "foo"
  zipcode : Integer = 42
  extras : Map of String to Integer = Map { "foo" : 42, "bar" : 69 }
end
```

3.10.11.2 Littéraux composites utilisables dans annotationValue et les valeurs d'enum

- List: List [1,2,3], List ["a","b"]

- **Map**: Map{"k1":1, "k2":2}
- **Record anonyme**: Record{name:"Alice", age:42}
- **Range**: Range -2048 .. 2048, Range 1..10
- **Json**: Json{"key": "value"}, Json[1,2,3], Json{"raw\": \"json\"}"
- **Geography**: Geography"POINT(2.3522 48.8566)", Geography{"type":"Point","coordinates":[2.3522,48.8566]}
- **Bytes**: Bytes"0xdeadbeef", Bytes[255, 0, 255], Bytes{"verified ": false, "algorithm": "sha256"}
- **Datetime**: Datetime"2021-01-01T00:00:00Z"
- **Date**: Date"2021-01-01"
- **Time**: Time"00:00:00"
- **Timestamp**: Timestamp"2021-01-01T00:00:00Z"
- **Sql**: Sql"SELECT * FROM users"

Exemple d'usage dans une enum:

```
enum Product(status: String, owners: List of String, address: Address, limits: Range) {
    active(status = "ok", owners = List["ops","qa"], address = Record{street:"A", zip: 12345})
}
```

Exemple d'usage avec des littéraux **Json**:

```
@ApiConfig (
    endpoint="https://api.example.com",
    headers=Json{"Content-Type": "application/json", "Authorization": "Bearer token"},
    schema=Json{"type": "object", "properties": {"name": {"type": "string"}}}
)

annotation ApiConfig is
    required endpoint : String
    optional headers : Json = Json{"Content-Type": "application/json"}
    optional schema : Json = Json{"type": "object"}
end
```

```
enum HttpStatus(code: Integer, details: Json) is (
    ok(code = 200, details = Json{"message": "Success", "data": {"count": 0}}),
    not_found(code = 404, details = Json{"error": "Not Found", "code": "E404"}),
    server_error(code = 500, details = Json{"error": "Internal Server Error", "stack": ""})
)
```

3.10.11.3 Littéraux Bytes Les littéraux Bytes permettent de représenter des données binaires de trois façons :

- **Chaîne hexadécimale** : Bytes"0xdeadbeef", Bytes"FFD8FFE0"
- **Tableau de valeurs numériques** : Bytes[255, 0, 255], Bytes[0x89, 0x50, 0x4E, 0x47]

- Les valeurs dans les tableaux peuvent être des nombres entiers (positifs ou négatifs) ou des valeurs hexadécimales.

```
@BinaryConfig(
    signature=Bytes("0xdeadbeef",
    hash=Bytes("a1b2c3d4e5f6",
    data=Bytes("U29tZSBiaW5hcnkgZGF0YQ==")
)
annotation BinaryConfig is
    required signature : Bytes
    optional hash : Bytes = Bytes("000000000000")
    optional data : Bytes = Bytes("SGVsbG8gV29ybGQ=")
end
```

```
enum FileType(extension: String, magic_bytes: Bytes, header: Bytes) is (
  pdf(extension = "pdf", magic_bytes = Bytes"25504446", header = Bytes"255044462D")
  jpeg(extension = "jpg", magic_bytes = Bytes"FFD8FFE0", header = Bytes"FFD8FFE000")
  png(extension = "png", magic_bytes = Bytes"89504E47", header = Bytes"89504E470D0A")
)
```

```
record ApiResponse is
  status : Integer
  data : Json = Json{}
  metadata : Json = Json{"timestamp": "2024-01-01T00:00:00Z", "version": "1.0"}
end
```

```
@LocationConfig(
    office=Geography"POINT(2.3522 48.8566)",
    coverage_area=Geography"POLYGON((2.3522 48.8566, 2.3523 48.8567, 2.3524 48.8568,
    delivery_route=Geography"LINESTRING(2.3522 48.8566, 2.3523 48.8567, 2.3524 48.8568,
    )
annotation LocationConfig is
    required office : Geography
    optional coverage_area : Geography = Geography"POINT(0.0 0.0)"
    optional delivery_route : Geography = Geography"LINESTRING(0.0 0.0, 1.0 1.0)"
end
```

```
enum CityType(name: String , location: Geography, bounds: Geography) is (
    paris(name = "Paris", location = Geography"POINT(2.3522 48.8566)", bounds = Geogr
    london(name = "London", location = Geography"POINT(0.1276 51.5074)", bounds = Ge
    tokyo(name = "Tokyo", location = Geography{"type":"Point","coordinates":[139.6917
)
```

```
record Store is
    name : String
    location : Geography
    service_area : Geography = Geography"POLYGON((0.0 0.0, 1.0 0.0, 1.0 1.0, 0.0 1.0,
    delivery_route : Geography = Geography"LINESTRING(0.0 0.0, 1.0 1.0)"
end
```

```
record Contact is
    name : String
    emails : List of String
    attributes: Map of String to String = Map{"role":"owner"}
end
```

```
record Team is
    name : String
    members : List of Contact
    quotas : Map of String to Range of Integer = Map{"daily": Range -100 .. 100}
end
```

```
annotation Project is
    required name : String
    optional team : Team
    optional labels : List of String = List["alpha","beta"]
    optional routing : Map of String to Map of String to Integer
end
```

```
@Project(
    name = "mesh",
    team = Record{
        name: "core",
        members: List[
            Record{name:"Alice", emails: List["a@example.com"]},
            Record{name:"Bob", emails: List["b@example.com","bob@work"]}
        ],

```

```

    quotas: Map{"daily": Range -50 .. 50}
  },
  routing = Map{
    "svcA": Map{"p": 80, "s": 20},
    "svcB": Map{"p": 60, "s": 40}
  }
)
module demo.advanced

```

3.11 Records et Collections

Les **Records** sont des structures de données composées qui permettent de regrouper des champs typés. Ils supportent la composition via les traits et peuvent contenir des **collections** (List, Map, Range).

3.11.1 Syntaxe des Records

```

record Contact is
  name : String
  email : String
  age : Integer = 0
end

// Record avec composition de traits
record User with Audit, Contactable is
  id : String
  role : String
end

```

3.11.2 Types de Collections

Meshr supporte quatre types de collections :

```

record UserProfile is
  tags : List of String = List["default", "user"]

```

```

    scores : List of Integer = List[85, 92, 78]
end

```

```

record Configuration is
    settings : Map of String to String = Map{"theme": "dark", "lang": "fr"}
    limits : Map of String to Integer = Map{"cpu": 80, "memory": 512}
end

```

```

record Metrics is
    score_range : Range of Integer = Range 0..100
    temperature : Range of Float = Range -40.0..60.0
end

```

```

@UserConfig(
    profile = Record{name: "Alice", age: 30},
    preferences = Record{theme: "dark", notifications: true}
)
annotation UserConfig is
    required profile : UserProfile
    required preferences : Preferences
end

```

3.11.3 Collections imbriquées

Les collections peuvent être imbriquées pour créer des structures complexes :

```

record ComplexData is
    // List de Maps
    configs : List of Map of String to String = List[
        Map{"env": "dev", "debug": "true"},
        Map{"env": "prod", "debug": "false"}
    ]

    // Map de Lists

```

```

categories : Map of String to List of String = Map{
  "colors": List["red", "green", "blue"],
  "sizes": List["small", "medium", "large"]
}

// Map de Ranges
limits : Map of String to Range of Integer = Map{
  "cpu": Range 0..100,
  "memory": Range 0..80
}
end

```

3.11.4 Usage dans tous les contextes

Les collections peuvent être utilisées partout où¹ un type est attendu :

```

@Project(
  tags = List["alpha", "beta"],
  config = Map{"timeout": 30, "retries": 3},
  range = Range 1..100
)
annotation Project is
  required tags : List of String
  required config : Map of String to Integer
  required range : Range of Integer
end

```

```

enum Status(
  code: Integer,
  tags: List of String,
  metadata: Map of String to String
) is (
  active(
    code = 200,
    tags = List["ok", "healthy"],

```

```

    metadata = Map{"type": "success", "level": "info"}
  )
)

```

```

trait WithMetadata is
  tags : List of String
  config : Map of String to String
  limits : Range of Integer
end

```

```

aspect DataQuality is
  metrics : List of String = List["accuracy", "completeness"]
  thresholds : Map of String to Float = Map{"accuracy": 0.95}
  score_range : Range of Integer = Range 0..100
end

```

3.11.5 Règles sémantiques

- Les collections sont **typées** : List of String, Map of String to Integer
- Les valeurs par défaut utilisent les **littéraux** : List ["a", "b"], Map{"k": "v"}
- Les collections peuvent être **imbriquées** : List of Map of String to String
- Les **références circulaires** sont interdites
- Les types dans les collections doivent être **définis** ou **importés**

4 8

5 Types personnalisés

Les **types personnalisés** permettent de créer des alias de types de base avec des contraintes spécifiques. Ils favorisent la réutilisabilité, la validation centralisée et l'expressivité du modèle de données.

5.1 Concept et objectifs

5.1.1 Principe de spécialisation

Les types personnalisés permettent de spécialiser les types de base (String, Integer, Float, etc.) avec des contraintes métier spécifiques, créant ainsi un vocabulaire typé riche et expressif.

5.1.2 Avantages

- **Réutilisabilité** : Définir une fois, utiliser partout
- **Validation centralisée** : Contraintes définies au niveau du type
- **Documentation intégrée** : Types auto-documentés avec sémantique métier
- **Sécurité** : Validation automatique des données
- **Expressivité** : Vocabulaire métier explicite

5.2 Syntaxe de base

5.2.1 Déclaration générale

```
type TypeName is BaseType constraint*
```

5.2.2 Types String avec contraintes

```
// Email avec pattern de validation
```

```
type Email is String pattern "^[^@]+@[^@]+\.[a-zA-Z]{2,}$"
```

```
// Code produit avec pattern et longueur
```

```
type ProductCode is String pattern "^[A-Z]{2}\\d{6}$" length 8..8
```

```
// Nom avec contrainte de longueur uniquement
```

```
type PersonName is String length 1..50
```

```
// URL avec pattern complexe
```

```
type URL is String pattern "https?://[\\w\\.-]+\\.[a-zA-Z]{2,}(/.*)?$"
```

5.2.3 Types numériques avec contraintes

```
// Age avec range et contrainte de positivité
type Age is Integer range 0..150 non_negative

// Prix avec range, précision et contrainte positive
type Price is Float range 0.01..999999.99 precision 2 positive

// Pourcentage avec range
type Percentage is Float range 0.0..100.0 non_negative

// Score avec range précis
type Score is Integer range 1..10 positive

// Température (peut être négative)
type Temperature is Float range -273.15..1000.0
```

5.2.4 Types temporels avec contraintes

```
// Date future
type FutureDate is Date after Date "2024-01-01"

// Heure de travail
type BusinessHour is Time between Time "08:00" and Time "18:00"

// Timestamp récent
type RecentTimestamp is Timestamp after Timestamp "2024-01-01T00:00:00Z"

// Date avec format personnalisé
type FormattedDate is Date format "DD/MM/YYYY"
```

5.2.5 Autres types spécialisés

```
// Données géographiques avec format
type GeoCoordinate is Geography format "geojson"

// Configuration JSON avec schéma
type ConfigData is Json schema "config-schema.json"

// Requête SQL formatée
type PostgreSQLQuery is Sql format "postgresql"
```

```
// Données binaires avec taille limit e
type SmallBinaryData is Bytes size 1..1024
```

5.3 Contraintes support es

5.3.1 Contraintes String

- pattern "regex" : Expression r guli re de validation
- length min..max : Contrainte de longueur

5.3.2 Contraintes num riques

- range min..max : Plage de valeurs autoris es
- precision n : Nombre de d cimales (Float/Double uniquement)
- positive : Valeurs strictement positives (> 0)
- negative : Valeurs strictement n gatives (< 0)
- non_negative : Valeurs positives ou nulles (>= 0)

5.3.3 Contraintes temporelles

- after date/timestamp : Post rieur   une date donn e
- before date/timestamp : Ant rieur   une date donn e
- between date1 and date2 : Dans un intervalle temporel
- format "format_string" : Format de date personnalis 

5.3.4 Contraintes sp cialis es

- format "format_string" : Format sp cifique (Geography, Sql, etc.)
- size min..max : Taille en bytes (Bytes uniquement)
- schema "schema_file" : Sch ma de validation (Json uniquement)

5.4 Utilisation dans les d clarations

5.4.1 Dans les entit s

```
entity Customer is
  id : String
  email : Email                // Validation automatique du pattern
  name : PersonName           // Contrainte de longueur
  age : Age                    // Range et non-n gatif
  website : URL                // Pattern URL valide
```

```

    satisfaction_score : Score          // 1-10
    discount_rate : Percentage          // 0-100%
    birth_date : Date
    next_contact_date : FutureDate     // Doit être dans le futur
    location : GeoCoordinate
    preferences : ConfigData
end

```

5.4.2 Dans les records

```

record ProductInfo is
    code : ProductCode                // Pattern et longueur validés
    name : PersonName
    price : Price                     // Precision et range
    discount : Percentage
    launch_date : FutureDate
end

```

5.4.3 Dans les relations

```

relation CustomerProduct is
    from Customer(id)
    to Product(id)

    purchase_date : RecentTimestamp
    paid_price : Price
    satisfaction : Score
end

```

5.4.4 Dans les mÃ©triques

```

metric CustomerSatisfactionAverage is
    source Customer
    calculation avg(satisfaction_score)

    filters {
        satisfaction_score > 0,
        age >= 18
    }
end

```

5.5 Types sealed

Les types personnalisés peuvent être marqués comme sealed pour empêcher leur extension :

```
sealed type RestrictedCode is String pattern "[A-Z]{3}$"
```

```
// Dans un autre module, cette extension serait interdite :
```

```
// type ExtendedCode is RestrictedCode length 4..4 // ERREUR
```

5.6 Règles sémantiques

5.6.1 Validation des contraintes

- Les contraintes doivent être compatibles avec le type de base
- Les contraintes multiples sont combinées avec un ET logique
- Les contraintes contradictoires génèrent une erreur de compilation

5.6.2 Résolution des types

- Les types personnalisés peuvent être utilisés partout où le type de base est attendu
- La validation des contraintes est effectuée automatiquement
- Les types personnalisés peuvent être importés et exportés comme les autres déclarations

5.6.3 Exemples d'erreurs

```
// ERREUR: Contrainte incompatible (pattern sur Integer)
type InvalidAge is Integer pattern "\\d+"
```

```
// ERREUR: Range invalide (min > max)
type InvalidRange is Integer range 100..50
```

```
// ERREUR: Precision sur Integer (pas supporté)
type InvalidPrecision is Integer precision 2
```

```
// ERREUR: Constraint temporelle sur String
type InvalidTemporal is String after Date "2024-01-01"
```

5.7 Bonnes pratiques

5.7.1 Nommage

- Utiliser **PascalCase** pour les noms de types : Email, ProductCode, BusinessHour
- Choisir des noms expressifs qui reflètent la sémantique métier

- Éviter les abréviations cryptiques

5.7.2 Organisation

- Grouper les types liés dans le même module
- Exporter les types réutilisables
- Documenter les types avec @Documented

5.7.3 Contraintes

- Définir des contraintes réalistes et testables
- Préférer des contraintes explicites aux contraintes implicites
- Tester les cas limites des contraintes

5.7.4 Exemple d'organisation

```
@Version("1.0.0")
@Documented(summary="Common business types for e-commerce domain")
module ecommerce.types
```

```
export { Email, ProductCode, Price, Percentage, Score }
```

```
@Documented(
    summary="Valid email address with domain validation",
    examples="user@company.com, admin@domain.org"
)
type Email is String pattern "^[^@]+@[^@]+\.[a-zA-Z]{2,}$"
```

```
@Documented(
    summary="Product code with standardized format",
    examples="AB123456, XY789012"
)
type ProductCode is String pattern "[A-Z]{2}\d{6}" length 8..8
```

7 Entités et Relations

Les **Entités** et **Relations** sont des artefacts déclaratifs pour la modélisation de données dans l'architecture Data-as-a-Product. Elles permettent de définir des structures de données persistantes et leurs interconnexions.

7.0.1 Entités

Une **entité** est un artefact fondamental de Meshr-lang qui représente une structure de données persistante dans l'architecture d'entreprise. Elle modélise les concepts métier centraux et leurs propriétés.

7.0.1.1 Concept et rôle Les entités servent à :

- **Modéliser les concepts métier** : Représenter les objets centraux du domaine (Customer, Product, Order, etc.)
- **Définir la structure des données** : Spécifier les champs, types, et contraintes
- **Assurer la cohérence** : Fournir un modèle de données unifié à travers l'organisation
- **Faciliter l'intégration** : Servir de contrat pour les APIs et les échanges de données
- **Documenter le domaine** : Exprimer explicitement les concepts et leurs relations

7.0.1.2 Caractéristiques des entités Une entité Meshr-lang possède :

- **Champs typés** : Chaque champ a un type explicite (String, Integer, Date, etc.)
- **Contraintes** : Validation des données via des patterns, ranges, ou contraintes personnalisées
- **Traits** : Comportements réutilisables appliqués à l'entité (audit, versioning, etc.)
- **Aspects** : Métadonnées et annotations spécifiques au contexte métier
- **Relations** : Connexions avec d'autres entités via des relations typées

```
entity <Name> [with <TraitName> (, <TraitName>)*] is
  <field declarations>
  [aspects { <AspectInstance>* }]
end
```

```
// Entité simple
entity Customer with WithAudit is
```

```

    CustomerId : String
    Email : String pattern "^[^@]+@[^@]+$"
    BirthDate : Date
    Phone : String length 10..15
end

// Entité avec aspects
entity SensitiveData with WithAudit is
    DataId : String
    Content : String
    Classification : String

    aspects {
        DataRetention { retention: Interval 5 year, steward: "dpo@example.com" }
    }
end

```

7.0.2 Types de Relations

Un **type de relation** est un modèle réutilisable décrivant les propriétés, contraintes et traits qu'une relation peut hériter.

```

type relation <Name> [with <TraitName> (, <TraitName>)*] is
    <field declarations>
    [aspects { <AspectInstance>* }]
end

```

```

type relation PLACED with WithAudit is
    since : Date
    channel : Channel
    quantity : Integer = 1
end

```

```

type relation FRIENDSHIP with WithAudit is
    since : Date

```

```

    note : String length 0..280
    closeness : Integer = 50
end

```

7.0.3 Relations

Une **relation** relie deux entités. Elle peut être non typée (définie ad hoc) ou typée (basée sur un type de relation), et peut être unidirectionnelle ou bidirectionnelle.

```

[bidirectional] relation <Name> [of type <RelationType>] [with <TraitName> (, <TraitName>)]
    from <Entity>(<Field>[, <Field>]*)
    to    <Entity>(<Field>[, <Field>]*)
    <field declarations>
    [aspects { <AspectInstance>* }]
end

```

```

// Relation non typée
relation CurrentAssignment is
    from Customer(SalesRepId)
    to   Employee(EmployeeId)

    since : Date
    note : String length 0..140
end

```

```

// Relation typée
relation ProductOrder of type PLACED is
    from Product(ProductId)
    to   Order(ProductId)

    discount : Float = 0.0
    special_instructions : String length 0..200
end

```

```

// Relation bidirectionnelle typée

```

```

bidirectional relation Friendship of type FRIENDSHIP is
  from Customer(CustomerId)
  to Customer(CustomerId)

  mutual_interests : List of String = List["shopping", "technology"]
end

```

7.0.4 Caractéristiques des Entités et Relations

- **Composition de traits** : Support de `with` pour injecter des comportements
 - **Aspects** : Instanciation d'aspects via `aspects { }`
 - **Modificateurs** : Support de `sealed` et `export`
 - **Annotations** : Métadonnées attachables via `@Annotation`
 - **Types de relations** : Réutilisation via `of type <RelationType>`
 - **Bidirectionnalité** : Relations symétriques via `bidirectional`
 - **Extrémités** : Définition des clés de jointure via `from/to`
-

7.1 Traits

Un **trait** est un mécanisme de composition et de réutilisation dans Meshr-lang qui permet de définir des blocs de déclarations (champs, métadonnées, contraintes, aspects) qui peuvent être injectés dans d'autres artefacts via la clause `with`. Les traits favorisent la factorisation, la composition déclarative, et l'élimination de la duplication de code.

7.1.1 Concept et rôle

Les traits servent à :

- **Factoriser le code** : Éviter la duplication de déclarations communes entre artefacts
- **Composer des comportements** : Combiner plusieurs traits pour créer des artefacts complexes
- **Standardiser les patterns** : Définir des conventions et patterns réutilisables

- **Maintenir la coh  rence** : Assurer que les m  mes propri  t  s sont d  finies de mani  re identique
- **Faciliter l'  volution** : Modifier un trait pour impacter tous les artefacts qui l'utilisent

7.1.2 Caract  ristiques des traits

Un trait Meshr-lang poss  de :

- **Champs typ  s** : D  finition de propri  t  s avec types et contraintes
- **Composition** : Possibilit   d'utiliser d'autres traits via with
- **Aspects int  gr  s** : Instanciation d'aspects qui seront h  rit  s
- **Injection** : Application aux artefacts via la clause with
- **R  utilisabilit  ** : Un m  me trait peut   tre utilis   par plusieurs artefacts

7.1.3 Diff  rence avec les aspects

Traits	Aspects
D��finissent des champs et comportements	D��finissent des m��tadonn��es et propri��t��s
Sont inject��s via with	Sont instanci��s via aspects { ... }
Ajoutent de la structure	Ajoutent du contexte
H��ritage de champs	H��ritage de m��tadonn��es

7.1.4 Syntaxe

```

trait <Identifiant> [with Base1, Base2] is
  <FieldName> : typeRef [= <valeur>]
  ...
  [aspects { <AspectInstance>* }]
end

```

7.1.5 Injection via with

La clause with permet d'injecter un ou plusieurs traits dans un artefact. Aujourd'hui, elle est supportée sur record et trait.

```
trait Audit is
  CreatedAt : Timestamp
  UpdatedAt : Timestamp
end
```

```
record Contact with Audit is
  Id : String
end
```

```
trait Contact is
  Email : String
end
```

```
trait ExtendedContact with Contact is
  Phone : String
end
```

```
record Person with Contact, Audit is
  Name : String
end
```

7.1.6 Aspects dans les Traits

Les traits peuvent contenir des **instanciations d'aspects** qui seront héritées par tous les artefacts qui utilisent le trait via with. Cela permet de factoriser les politiques transversales.

```
// Définition des aspects
aspect DataRetention is
  retention : Interval
  steward : String
end
```

```
aspect SecurityLevel is
  level : String = "public"
  encryption_required : Boolean = false
```

end

```
// Trait avec aspects
trait WithSecurity is
  access_level : String
  permissions : List of String

  aspects {
    SecurityLevel { level: "restricted", encryption_required: true },
    DataRetention { retention: Interval 5 year, steward: "security@example.com" }
  }
end
```

```
// Utilisation du trait avec ses aspects
record User with WithSecurity is
  id : String
  name : String
end
```

**** Héritage des Aspects : - Les aspects définis dans un trait sont automatiquement appliqués****
aux artefacts qui utilisent ce trait - Les aspects peuvent être **redéfinis** au niveau de l'artefact consommateur si nécessaire - Les règles de **composition des aspects** s'appliquent normalement (intersection, explicitation des conflits)

7.1.7 Règles de composition (sémantique)

- Monotonie: on ne relâche pas les exigences (required $\square$ optional interdit, suppression de contraintes interdite).
- Intersection: des contraintes compatibles s'additionnent; on garde la version la plus restrictive.
- Explicitation: en cas de conflit dur (types différents, contraintes incompatibles, aspects contradictoires), l'artefact consommateur doit redéfinir explicitement.

Récapitulatif:

Situation	Exemple	Résolution
Compatibilité parfaite	X: string + X: string	Fusion automatique

Situation	Exemple	R�solution
Extension compatible	Y: string + Y: string required	required lemporte
Contraintes compatibles	Z: string + Z: string pattern "[A-Z]+\$"	Conserve la contrainte (intersection)
Conflit (types)	A: int + A: string	Erreur □ red�finir dans l'artefact
Conflit (contraintes)	B: string length 1..10 + B: string length 20..30	Erreur □ red�finir
Conflit (aspects)	C aspects { PII { level : high } } + C aspects { PII { level : low } }	Erreur □ red�finir

Notes: - Ces r gles sont v rifi es par les outils (validation s mantique), pas   l'analyse syntaxique.
- typeRef est autoris  dans les champs de trait , de record, dans les attributs d' num et dans les champs d'annotations.

7.1.8 [2025-09-09] Module = unit , namespace et fichier unique

- **Contexte** : Il fallait d cider si l'on autorisait plusieurs modules par fichier, et si y avait un lien fort avec l'arborescence.
- **D cision** : Le langage impose qu'un fichier .meshr d clare **un seul module**. Le nom du module est un **namespace**. L'arborescence n'est **pas contrainte**, mais une convention est propos e.
- **Pourquoi** : Cela permet une r solution simple par les outils (CLI, LSP),  vite les collisions de symboles, et reste lisible.

7.2 Aspects

Un **Aspect** est un m canisme de m tadonn es structur es dans Meshr-lang qui permet d'enrichir et d'annoter les artefacts avec des informations contextuelles sp cifiques au domaine m tier. Contrairement aux traits qui d finissent des comportements, les aspects se concentrent sur la description de propri t s, contraintes, et m tadonn es.

7.2.1 Concept et rôle

Les aspects servent à :

- **Enrichir les métadonnées** : Ajouter des informations contextuelles aux artefacts (gouvernance, qualité, sécurité)
- **Définir des politiques** : Exprimer des règles métier et des contraintes de conformité
- **Documenter le contexte** : Capturer des informations sur l'origine, l'usage, et la signification des données
- **Faciliter la gouvernance** : Intégrer les exigences réglementaires et les bonnes pratiques
- **Améliorer la traçabilité** : Associer des informations de provenance et de lineage

7.2.2 Caractéristiques des aspects

Un aspect Meshr-lang possède :

- **Champs typés** : Propriétés avec types explicites et contraintes optionnelles
- **Valeurs par défaut** : Définition de valeurs par défaut pour les champs optionnels
- **Héritage** : Possibilité d'étendre d'autres aspects pour la réutilisation
- **Instanciation** : Application aux artefacts via des blocs aspects { ... }
- **Composition** : Combinaison de plusieurs aspects sur un même artefact

7.2.3 Syntaxe de base

```
// Aspect simple
aspect BusinessDesc is
  domain : String
  summary : String
end

// Aspect abstrait
abstract aspect GovernanceBase is
  steward : String
  policy_id : String
end
```

```
// Aspect avec hÃ©ritage
aspect DataRetention extends GovernanceBase is
  retention : Interval
end

// Aspect avec composition (traits)
aspect DataQuality extends GovernanceBase with WithLifecycle is
  quality_score : Float
  validation_rules : List of String
end
```

7.2.4 HÃ©ritage et composition

Les Aspects supportent : - **HÃ©ritage simple** : extends AspectName - **Composition** : with TraitName (mÃªme syntaxe que les records/traits) - **Combinaison** : extends AspectName with Trait1, Trait2

```
trait WithLifecycle is
  created_at : Timestamp
  updated_at : Timestamp
end
```

```
abstract aspect GovernanceBase with WithLifecycle is
  steward : String
  policy_id : String
end
```

```
aspect DataRetention extends GovernanceBase is
  retention : Interval
end
```

7.2.5 Types d'Aspects

- **Aspects abstraits** : abstract aspect - dÃ©finitions gÃ©nÃ©riques non instanciables
- **Aspects concrets** : aspect - instanciables dans les artefacts

7.2.6 Champs et contraintes

Les Aspects supportent tous les types et contraintes disponibles :

```
aspect ComprehensiveMetadata is
  // Types de base avec contraintes
  name : String length 1..100
  email : String pattern "^[^@]+@[^@]+$"

  // Types composites
  tags : List of String = List["default"]
  metadata : Map of String to String = Map{"source":"manual"}
  range : Range of Integer = Range 0..100

  // Types temporels
  created : Timestamp
  duration : Interval

  // Valeurs par défaut
  active : Boolean = true
  count : Integer = 0
end
```

7.2.7 Annotations sur les Aspects

Les Aspects peuvent être annotés comme tous les autres artefacts :

```
@since("1.0.0")
@id("business-desc")
@display_name("Business Description")
@description("Un aspect permettant d'enrichir les champs avec un domaine métier et
aspect BusinessDesc is
  domain : String
  summary : String
end
```

7.2.8 Règles de composition

Les mêmes règles que pour les Traits s'appliquent lors de la fusion de définitions :

- **Fusion automatique** si signatures identiques
- **Renforcement** si extension compatible (ex. optional $\square$ required)
- **Intersection** des contraintes si compatibles
- **Erreur explicite** si conflit strict (type, contrainte, aspect key clash)

7.2.9 ® Instantiation (future)

Les Aspects seront instanciés avec le mot-clé `aspects { ... }` au niveau d'un artefact ou d'un champ :

```
// Syntaxe future pour l'instanciation
entity Customer is
  CustomerId : String
  Email      : String
  aspects {
    DataRetention {
      retention : Interval 1 year ,
      steward   : "dpo@example.com",
      policy_id : "GDPR-001"
    }
  }
end
```

9 Policies

Les **policies** (politiques) permettent de d finir des r gles de gouvernance et de validation automatiques sur les d clarations Meshr-Lang. Elles constituent un m canisme puissant pour assurer la conformit , la qualit  et la coh rence des mod les de donn es.

9.1 Concept et objectifs

9.1.1 Principe de gouvernance d clarative

Les policies permettent d'exprimer des r gles m tier et de conformit  directement dans le mod le, avec application automatique et actions correctives.

9.1.2 Domaines d'application

- **Gouvernance des donn es** : Validation des aspects obligatoires
- **Conformit ** : Respect des r glementations (GDPR, SOX, etc.)
- **Qualit ** : Validation des contraintes et bonnes pratiques
- **S curit ** : Application automatique des politiques de s curit 
- **Maintenance** : Corrections automatiques et notifications

9.2 Syntaxe de base

9.2.1 Structure g n rale

```
policy PolicyName is
  verbatim "Description libre optionnelle"

  scope "entity"|"record"|"trait"|"aspect"|"relation"|"metric"|"type"

  // Option A : Condition simple
  condition missing aspect { AspectName }
  condition exists aspect { AspectName }

  // Option B : Pattern matching (pour cas complexes)
  match scope_type is
    { pattern1 } -> actions { ... },
    { pattern2 } or { pattern3 } -> actions { ... }
  end
```

```

actions {
  deny with message "Error message",
  warn with message "Warning message",
  notify "email@company.com" with message "Notification",
  patch modify aspect AspectName { property == value } with message "Patch message"
}
end

```

9.3 Composants détaillés

9.3.1 1. Verbatim (optionnel)

Documentation libre en langage naturel, placée immédiatement après is.

```

policy DataGovernanceBasic is
  verbatim "All entities with personal data must have proper governance aspects"

  scope "entity"
    // ...
end

```

9.3.2 2. Scope (obligatoire)

Définit le type de déclaration sur lequel la policy s'applique.

```

// Scopes supportés
scope "entity"      // Entités
scope "record"      // Records
scope "trait"       // Traits
scope "aspect"      // Aspects
scope "relation"    // Relations
scope "metric"      // Métriques
scope "type"        // Types personnalisés

// Scope avec filtre
scope "entity" where contains_personal_data == true
scope "metric" where business_criticality == "high"

```

9.3.3 3. Conditions

Expressions déclaratives pour tester l'état des déclarations.

```

// Aspects manquants
condition missing aspect { DataClassification }
condition missing aspect { GDPRCompliance, DataStewardship }

// Aspects existants
condition exists aspect { DataClassification }

// Propriétés d'aspects
condition aspects.DataClassification.level == "confidential"
condition aspects.GDPRCompliance.retention_period > Interval 5 year

```

9.3.4 4. Pattern Matching

Pour des validations complexes, utilisez la syntaxe des motifs.

```

match table is
  { partitioning is null } ->
    actions { deny with message "Partitioning required" }
end

```

```

match table is
  { partitioning is null } or { expiration is null } ->
    actions { deny with message "Both partitioning and expiration required" }
end

```

```

match dataset is
  { expiration < 30 } ->
    actions { deny with message "Retention too short" },
  { expiration > 365 } ->
    actions { deny with message "Retention too long" }
end

```

```

match dataset is
  { label "env" in { "dev", "staging" } } ->
    actions { warn with message "Non-production environment" }
end

```

9.3.5 5. Actions

Définissent les mesures à prendre quand une policy est violée.

```

actions {
  deny with message "Blocking error message",
  warn with message "Warning message"
}

```

```

actions {
  notify "admin@company.com" with message "Governance violation detected",
  notify "dpo@company.com" with message "GDPR compliance issue"
}

```

```

actions {
  // Ajouter un aspect manquant
  patch add aspect DataClassification {
    level == "internal",
    steward == "data.team@company.com"
  } with message "Added default classification",

  // Modifier un aspect existant
  patch modify aspect GDPRCompliance {
    retention_period == Interval 7 year
  } with message "Updated retention period",

  // Supprimer un aspect obsolète
  patch remove aspect DeprecatedAspect with message "Removed deprecated aspect"
}

```

9.4 Exemples complets

9.4.1 Policy de gouvernance basique

```
policy RequireDataClassification is
  verbatim "All entities must have data classification for governance"

  scope "entity"
    condition missing aspect { DataClassification }

    actions {
      deny with message "Entity must have DataClassification aspect"
    }
  end
end
```

9.4.2 Policy avec validation d'aspects multiples

```
policy GDPRCompliance is
  verbatim "Entities with personal data need GDPR compliance"

  scope "entity" where contains_personal_data == true
    condition missing aspect { GDPRCompliance, DataClassification }

    actions {
      patch add aspect GDPRCompliance {
        lawful_basis == "consent",
        retention_period == Interval 7 year,
        data_subject_rights == List["access", "rectification", "erasure"]
      } with message "Auto-added GDPR compliance",

      notify "dpo@company.com" with message "New entity with personal data created"
    }
  end
end
```

9.4.3 Policy avec pattern matching avancé

```
policy SensitiveDataValidation is
  verbatim "Sensitive columns must be properly tagged and encrypted"

  scope "column"

  match column is
    { (name like "ssn" or name like "iban" or name like "credit_card") and not exists
      actions {
        deny with message "Sensitive columns must be tagged",
        patch add policy_tag "pii.sensitive" with message "Auto-tagged sensitive column"
      },
    { name like "email" and constraints missing pattern } ->
      actions {
        warn with message "Email fields should have pattern validation",
        patch add constraint pattern "^[^@]+@[^@]+$" with message "Added email pattern"
      }
  end
end
```

9.4.4 Policy de qualité des métriques

```
policy MetricQualityStandards is
  verbatim "All business-critical metrics must have proper governance"

  scope "metric"
  condition missing aspect { MetricGovernance }

  actions {
    patch add aspect MetricGovernance {
      business_owner == "data.owner@company.com",
      technical_owner == "analytics@company.com",
      refresh_frequency == Interval 1 day,
      accuracy_threshold == 95.0,
      business_criticality == "medium"
    } with message "Added default metric governance",

    warn with message "Metric should have explicit governance metadata"
```

```
}  
end
```

9.5 Règles sémantiques

9.5.1 Validation des scopes

- Les scopes doivent correspondre aux types de déclarations Meshr-Lang
- Les filtres where utilisent les expressions standard du langage
- Les références aux aspects doivent être valides et importées

9.5.2 Conditions vs Pattern Matching

- **Conditions** : Pour des tests simples sur les aspects
- **Pattern Matching** : Pour des validations complexes avec multiple branches
- Les deux sont mutuellement exclusifs dans une même policy

9.5.3 Actions et effets de bord

- **deny** : Bloque la compilation/validation
- **warn** : Émet un avertissement non-bloquant
- **notify** : Envoie une notification (email, webhook, etc.)
- **patch** : Modifie automatiquement le module

9.5.4 Patch operations

- **add aspect** : Ajoute un aspect complet (erreur si existe déjà)
- **modify aspect** : Modifie des propriétés d'un aspect existant
- **remove aspect** : Supprime un aspect complet

9.6 Bonnes pratiques

9.6.1 Organisation

- Grouper les politiques liées dans des modules dédiés
- Utiliser des noms explicites : RequireGDPRCompliance, ValidateDataClassification
- Documenter avec verbatim les politiques complexes

9.6.2 Messages

- Messages explicites et actionnables
- Références aux politiques d'entreprise quand approprié
- Messages de patch pour traçabilité des modifications automatiques

9.6.3 Exemple d'organisation

```
@Version("1.0.0")
```

```
@Documented(summary="Data governance policies for customer domain")  
module customer.policies
```

```
import { DataClassification, GDPRCompliance } from meshr.security  
import { DataStewardship } from meshr.governance
```

```
export { RequireDataClassification, GDPRValidation, StewardshipPolicy }
```

```
policy RequireDataClassification is  
  verbatim "All customer entities must have data classification"
```

```
  scope "entity"  
    condition missing aspect { DataClassification }
```

```
    actions {  
      deny with message "Customer entities require data classification per policy GOV"  
    }  
  end
```

10 13

11 Modificateur sealed

Le modificateur sealed permet de **verrouiller l'extension** d'un artefact. Par défaut, tous les artefacts sont **ouverts** (extensibles). Un artefact marqué sealed ne peut plus être hérité, étendu ou redéfini partiellement. L'usage par référence ou instanciation **reste autorisé**.

11.0.1 Syntaxe

Le modificateur sealed peut précéder : - sealed enum - sealed record - sealed trait - sealed aspect

Les **annotations** ne peuvent pas être sealed. Toute tentative doit produire une erreur.

11.0.2 Règles d'héritage et d'extension

```
sealed enum Color is ("red", "green", "blue")
```

- **INTERDIT** : étendre ou redéfinir Color
- **AUTORISÉ** : utiliser Color comme type de champ

```
sealed record Address is
  street : String
  city   : String
end
```

- **INTERDIT** : hériter de Address
- **AUTORISÉ** : l'utiliser comme type

```
sealed trait WithAudit is
  created_at : Timestamp
  updated_at : Timestamp
end
```

- **INTERDIT** : créer trait ExtendedAudit extends WithAudit
- **AUTORISÉ** : utiliser WithAudit via with

```
sealed aspect Governance is
  steward    : String
  policy_id  : String
end
```

- **INTERDIT** : aspect AdvancedGovernance extends Governance
- **AUTORISÉ** : instancier Governance dans un bloc aspects { ... }

11.0.3 Exemples valides

```
sealed enum Color is ("red", "green", "blue")
```

```
sealed record Address is
  street : String
  city   : String
end
```

```
sealed trait WithAudit is
  created_at : Timestamp
  updated_at : Timestamp
end
```

```
sealed aspect Governance is
  steward    : String
  policy_id  : String
end
```

```
record User is
  name      : String
  address   : Address // Usage autorisé d'un record sealed
  color     : Color   // Usage autorisé d'un enum sealed
end
```

```
record AuditLog with WithAudit is // Usage autoris  d'un trait sealed
  action : String
end
```

11.0.4 Exemples invalides

```
// enum scell  ne peut pas  tre  tendue
enum ExtendedColor extends Color is ("yellow")
```

```
// record scell  ne peut pas  tre h rit 
record FullAddress extends Address is
  country : String
end
```

```
// trait scell  ne peut pas  tre h rit 
trait ExtendedAudit extends WithAudit is
  deleted_at : Timestamp
end
```

```
// aspect scell  ne peut pas  tre h rit 
aspect AdvancedGovernance extends Governance is
  extra : String
end
```

```
// annotation ne peut pas  tre sealed
sealed annotation InvalidAnnotation is
  required name : String
end
```

11.0.5 R gles s mantiques

- Un artefact sealed **ne peut pas  tre  tendu** via extends
- Un artefact sealed **peut  tre utilis ** comme type de r f rence
- Un trait sealed **peut  tre compos ** via with
- Un aspect sealed **peut  tre instanci ** dans des blocs aspects
- Les **annotations ne peuvent jamais  tre sealed**
- L'ordre des modificateurs est : sealed abstract aspect (pas abstract sealed)

12 MÃtriques (Metrics)

12.1 Introduction

Les **mÃtriques** permettent de dÃfinir des KPIs (Key Performance Indicators) et des mesures mÃtier directement dans le modÃle sÃmantique. Elles s'intÃgrent naturellement avec les entitÃs, relations et aspects existants pour gÃnÃrer automatiquement des tableaux de bord, des requÃtes SQL et des pipelines de donnÃes.

Important : Les mots-clÃs des mÃtriques (metric, source, calculation, aggregation, unit, outputs, dimensions, filters, temporal, window, refresh_frequency, historical_depth, match, or) sont **rÃservÃs** et ne peuvent pas Ãtre utilisÃs comme noms de champs. Voir la section des mots-clÃs rÃservÃs pour la liste complÃte.

12.1.1 Philosophie

- **DÃclaratif** : DÃcrire QUOI mesurer, pas COMMENT l'implÃmenter
- **RÃutilisable** : S'appuie sur le modÃle sÃmantique existant
- **GouvernÃ** : Aspects de qualitÃ, sÃcuritÃ et lineage intÃgrÃs
- **Portable** : GÃnÃration vers BigQuery, Snowflake, dbt, etc.

12.2 Syntaxe de base

12.2.1 Structure minimale

```
metric MetricName is
  source EntityName
  calculation expression
  unit "unit_string"
end
```

12.2.2 Exemple simple

```
@Documented(summary="Temps moyen de recrutement")
metric TimeToHire is
  source Application
  calculation avg(offer_date - application_date)
  unit "days"
end
```

12.3 Composants détaillés

12.3.1 1. Source de données

Syntaxe : source EntityName

```
source Application    // Entité source des données
source Customer      // Autre exemple
```

La source doit être une entité déclarée dans le modèle.

12.3.2 2. Calcul principal

Syntaxe : calculation expression

```
calculation avg(offer_date - application_date)
calculation sum(related(Order).total_amount)
calculation count(*) where status == ApplicationStatus.hired
```

Fonctions supportées : - **Aggrégation :** avg(), sum(), count(), min(), max() - **Temporelles :** date_trunc(), date_part() - **Relationnelles :** related(Entity, field)

12.3.3 3. Unité de mesure

Syntaxe : unit "string"

```

unit "days"
unit "EUR"
unit "percentage"
unit "score"

```

12.3.4 4. Outputs (optionnel)

Défini les champs de sortie de la matrice :

```

outputs {
  average_days : Float ,
  median_days : Float ,
  sample_size : Integer ,
  calculation_timestamp : Timestamp
}

```

12.3.5 5. Dimensions (optionnel)

Défini les axes d'analyse pour la matrice :

```

dimensions {
  department related(JobPosting , job_id).department_id ,
  period date_trunc("month", application_date)
}

```

```

dimensions {
  salary_tier match salary_offered is
    0..49999 -> "entry level",
    50000..79999 -> "mid level",
    80000.. -> "senior level"
  end,

  channel_type match source_channel is
    SourceChannel.linkedin or SourceChannel."social media" -> "digital",
    SourceChannel."internal referral" -> "internal",
    _ -> "external"
  end
}

```

```
}
```

Patterns support : - **Ranges** : 0..49999, 50000.. (open-ended) - **Enums** : ApplicationStatus.hired
- **OR patterns** : pattern1 or pattern2 - **Wildcard** : _ (catch-all)

12.3.6 6. Filtres (optionnel)

Conditions de filtrage des données :

```
filters {  
  application_status == ApplicationStatus.hired ,  
  application_date >= Date "2024-01-01",  
  department_id in List["eng", "product"]  
}
```

12.3.7 7. Agrégation avancée (optionnel)

Pour des métriques complexes avec plusieurs calculs :

```
aggregation {  
  recruitment_costs as sum(related(RecruitmentCost, application_id).amount),  
  total_hires as count(*) where application_status == ApplicationStatus.hired ,  
  cost_per_hire as recruitment_costs / total_hires  
}
```

12.3.8 8. Configuration temporelle (optionnel)

```
temporal {  
  window Interval 6 month ,  
  refresh_frequency Interval 1 day ,  
  historical_depth Interval 2 year  
}
```

12.3.9 9. Aspects (optionnel)

Métadonnées de gouvernance :

```

aspects {
  MetricGovernance {
    business_owner: "hr.director@company.com",
    business_criticality: "high"
  },
  DataClassification {
    level: "internal",
    access_control: "role-based"
  }
}

```

12.4 Exemple complet

```
@BusinessCritical(impact="high", stakeholders=["hr director", "ceo"])
```

```
@Documented(summary="Temps moyen de recrutement par dÃpartement")
```

```
metric TimeToHireByDepartment is
```

```
  source Application
```

```
  calculation avg(offer_date - application_date)
```

```
  unit "days"
```

```
  outputs {
```

```
    average_days : Float,
```

```
    median_days : Float,
```

```
    sample_size : Integer
```

```
  }
```

```
  dimensions {
```

```
    department related(JobPosting, job_id).department_id,
```

```
    seniority_tier match related(JobPosting, job_id).seniority_level is
```

```
      SeniorityLevel.intern or SeniorityLevel.junior -> "junior",
```

```
      SeniorityLevel.senior or SeniorityLevel.lead -> "senior",
```

```
      _ -> "executive"
```

```
    end,
```

```
    period date_trunc("month", application_date)
```

```
  }
```

```
  filters {
```

```
    application_status == ApplicationStatus.hired,
```

```
    application_date >= Date "2024-01-01"
```

```

}

temporal {
  window Interval 6 month,
  refresh_frequency Interval 1 day
}

aspects {
  MetricGovernance {
    business_owner: "hr.director@company.com",
    technical_owner: "hr.analytics@company.com",
    business_criticality: "high"
  }
}
end

```

12.5 Règles sémantiques

12.5.1 Obligatoires

- **source** : Entité source (doit exister dans le modèle)
- **calculation OU aggregation** : Au moins un des deux
- **unit** : Unité de mesure

12.5.2 Optionnels

- Tous les autres blocs peuvent être omis

12.5.3 Validation

- **Types** : Cohérence des types dans les expressions
- **Relations** : Validation des chemins `related( Entity , field )`
- **Enums** : Validation des valeurs `EnumName.value`
- **Patterns** : Exhaustivité recommandée (warning si pas de `_`)

12.6 Artefacts d'Ãfinissables

12.6.1 1. domain

DÃclaration dun domaine hiÃrarchique.

```
domain retail.marketing {  
  description = "Marketing domain for the retail perimeter"  
}
```

12.6.2 2. product

DÃclaration dun produit de donnÃes.

```
product leads_b2c_import {  
  domain = retail.marketing  
  contracts = [  
    contract:marketing:leads_b2c_import:1.0.0  
  ]  
}
```

12.6.3 3. contract

(DÃfini dans une autre couche ou gÃnÃrÃ via DCDL)

12.6.4 4. aspect

(TBD)

12.6.5 5. team

(TBD)

12.6.6 6. policy

(TBD)

12.7 Décisions de conception

12.7.1 [2025-09-09] Syntaxe déclarative avec blocs { }

- **Contexte** : Le langage doit rester simple, lisible pour les métiers, mais formalisable en grammaire ANTLR.
- **Décision** : Utiliser une syntaxe à blocs ({}), proche de HCL/Terraform.
- **Pourquoi** : Meilleure lisibilité et extensibilité que YAML/JSON, tout en étant facilement parsable.

12.7.2 [2025-09-10] Visibilité par export explicite

- **Contexte** : Le langage devait gérer la visibilité entre modules pour encourager l'encapsulation.
- **Décision** : Un artefact déclaré dans un module nest accessible depuis l'extérieur que s'il est précédé du mot-clé export.
- **Pourquoi** : Cette règle encourage une séparation claire entre API publique et éléments internes, facilite la documentation automatique et le linting.

12.7.3 [2025-09-10] Deux formes d'export : inline ou groupé

- **Contexte** : Besoin de contrôler la visibilité des artefacts à l'extérieur du module.
- **Décision** : Deux formes sont supportées : inline (export decl) et groupé (export { A, B }).

- **Justification** : Favorise à la fois la lisibilité locale (inline) et la gestion explicite de l'API publique (groupes après les imports).

12.7.4 [2025-09-12] Modificateur `sealed` pour contrôler l'extensibilité

- **Contexte** : Besoin de verrouiller l'extension de certains artefacts pour garantir la stabilité de l'API et éviter les dérives non contrôlées.
- **Décision** : Introduction du modificateur `sealed` applicable aux `enum`, `record`, `trait` et `aspect`. Les annotations ne peuvent pas être `sealed`.
- **Pourquoi** : Permet de définir des contrats stables (ex: `enums` de statut, `records` d'adresse) tout en conservant la flexibilité de composition via `with` pour les traits.

12.7.5 [2025-09-12] Types de collections typés et littéraux

- **Contexte** : Besoin de structures de données complexes pour représenter des métadonnées, configurations et données structurées dans les artefacts Data Mesh.
- **Décision** : Introduction de quatre types de collections : `List of T`, `Map of K to V`, `Range of T`, et `Record anonyme`, avec des littéraux syntaxiques (`List [...]`, `Map{...}`, `Range a..b`, `Record{...}`).
- **Pourquoi** : Permet une expressivité maximale pour les métadonnées complexes tout en maintenant la sécurité des types et la lisibilité du code. Les collections peuvent être imbriquées et utilisées dans tous les contextes (annotations, `enums`, `records`, traits, aspects).

12.7.6 [2025-09-12] Entités et Relations pour la modélisation de données

- **Contexte** : Besoin de modéliser des structures de données persistantes et leurs interconnexions dans l'architecture Data-as-a-Product, avec support des relations complexes et de la gouvernance des données.
- **Décision** : Introduction de trois nouveaux artefacts : `entity` (structures de données persistantes), `type relation` (modèles réutilisables de relations), et `relation` (connexions entre entités avec support bidirectionnel).
- **Pourquoi** : Permet une modélisation complète du domaine métier avec support des relations complexes, de la gouvernance via les aspects, et de la réutilisabilité via les types de relations. Les entités et relations supportent tous les modificateurs existants (`sealed`, `export`, annotations) et s'intègrent parfaitement avec le système de traits et d'aspects.

12.8 Bibliothèque Standard (StdLib)

La **bibliothèque standard** de Meshr-Lang fournit un ensemble d'artefacts prêts à l'emploi, d'aspects et de types communs pour faciliter le développement d'architectures Data-as-a-Product. Elle est organisée en modules thématiques et est disponible dans le dossier `stdlib` /.

12.8.1 Objectifs

- **Réutilisabilité** : Composants standards pour les cas d'usage courants
- **Cohérence** : Conventions et patterns établis
- **Productivité** : Réduction du temps de développement
- **Qualité** : Composants testés et validés

12.8.2 Modules disponibles

```
// Aspects de sécurité
aspect DataClassification is
  level : String // "public", "internal", "confidential", "restricted"
  encryption_required : Boolean = false
  access_control : String = "role-based"
end

aspect GDPRCompliance is
  data_subject_rights : List of String
  retention_period : Interval
  lawful_basis : String
  dpo_contact : String
end

// Types de relations de sécurité
type relation ACCESS_CONTROL is
  granted_by : String
```

```
    granted_at : Timestamp
    expires_at : Timestamp
    permissions : List of String
end
```

```
// Aspects de gouvernance
aspect DataStewardship is
    steward : String
    owner : String
    business_domain : String
    last_review : Date
end
```

```
aspect DataQuality is
    quality_score : Float range 0.0..1.0
    validation_rules : List of String
    monitoring_frequency : Interval
    alert_threshold : Float
end
```

```
// Traits de gouvernance
trait WithStewardship is
    steward : String
    owner : String

    aspects {
        DataStewardship { steward: steward , owner: owner, business_domain: "default" }
    }
end
```

```
// Types métier communs
enum BusinessDomain is (
    "finance",
    "marketing",
    "sales",
    "hr",
```

```

    "operations",
    "product"
)

enum DataSensitivity is (
    "public",
    "internal",
    "confidential",
    "restricted"
)

// Traits métier
trait WithBusinessContext is
    domain : BusinessDomain
    sensitivity : DataSensitivity
    business_owner : String
end

// Aspects de cycle de vie
aspect DataLifecycle is
    created_at : Timestamp
    updated_at : Timestamp
    version : String
    status : String // "draft", "active", "deprecated", "archived"
    deprecation_date : Date
end

// Traits de cycle de vie
trait WithLifecycle is
    created_at : Timestamp
    updated_at : Timestamp
    version : String = "1.0.0"

    aspects {
        DataLifecycle {
            created_at: created_at,
            updated_at: updated_at,
            version: version,

```

```

        status: "active"
    }
}
end

```

```

// MÃta-annotations
@Target("annotation")
@Retention(model)
annotation Target is
    required value : AnnotationTarget
end

```

```

@Target("annotation")
@Retention(model)
annotation Retention is
    required value : RetentionPolicy
end

```

```

@Target("annotation")
@Retention(model)
annotation Repeatable is
    optional value : Boolean = true
end

```

```

// Annotations communes
@Target("all")
@Retention(model)
annotation Experimental is
    optional reason : String = "Feature is experimental and may change"
end

```

```

@Target("all")
@Retention(model)
annotation Deprecated is
    required reason : String
    optional since : String = ""
    optional replacement : String = ""
end

```

```

@Target(" all ")
@Retention(model)
annotation Version is
    required value : String
end

```

```

@Target(" all ")
@Retention(model)
annotation Author is
    required name : String
    optional email : String = ""
    optional organization : String = ""
end

```

```

@Target(" all ")
@Retention(model)
annotation Scope is
    required level : ScopeLevel
    optional description : String = ""
end

```

```

@Target(" all ")
@Retention(model)
annotation Confidentiality is
    required level : ConfidentialityLevel
    optional classification : String = ""
    optional handling_instructions : String = ""
end

```

```

@Target(" all ")
@Retention(model)
annotation Documented is
    required summary : String
    optional description : String = ""
    optional examples : String = ""
    optional see_also : String = ""
end

```

```
// Types de base et communs
```

```
enum Status is (  
  "active",  
  "inactive",  
  "pending",  
  "suspended",  
  "archived"  
)
```

```
enum Priority is (  
  "low",  
  "medium",  
  "high",  
  "critical"  
)
```

```
aspect ContactInfo is  
  email : String  
  phone : String  
  mobile : String  
  website : String  
end
```

```
aspect Address is  
  street : String  
  city : String  
  postal_code : String  
  country : String  
end
```

```
trait WithContactInfo is  
  email : String  
  phone : String
```

```
aspects {  
  ContactInfo {  
    email: email,  
    phone: phone,  
    mobile: "",  
    website: ""
```

```

    }
  }
end

```

```

// Types pour l'analytics
enum MetricType is (
  "counter",
  "gauge",
  "histogram",
  "summary"
)

```

```

type relation DATA_LINEAGE is
  source_system : String
  transformation : String
  frequency : Interval
  last_updated : Timestamp
end

```

```

// Aspects d'analytics
aspect DataLineage is
  source_systems : List of String
  transformations : List of String
  refresh_frequency : Interval
  sla : Interval
end

```

12.8.3 Structure actuelle

```

stdlib /
  core /                                # Modules fondamentaux
  □ security.meshr                      # Aspects et types de sécurité
  □ governance.meshr                   # Gouvernance des données
  □ lifecycle.meshr                    # Cycle de vie des données
  □ types.meshr                        # Types de base et communs
  □ annotations.meshr                  # Annotations standard et méta-annotations
  examples /                           # Exemples d'utilisation

```

- simple-customer.meshr
- customer-entity.meshr
- simple-imports.meshr
- imports-demo.meshr
- annotations-usage.meshr
- meta-annotations.meshr
- business / # Modules métier (À venir)
- domains.meshr
- entities.meshr
- processes.meshr
- analytics / # Modules d'analytics (À venir)
- metrics.meshr
- lineage.meshr
- quality.meshr
- integrations / # Intégrations réglementaires (À venir)
- gdpr.meshr
- sox.meshr
- iso27001.meshr

12.8.4 Cas d'usage

```
// Exemple d'utilisation de la stdlib
import { DataClassification, GDPRCompliance, WithSecurity } from meshr.security
import { DataStewardship, WithStewardship } from meshr.governance
import { WithLifecycle } from meshr.lifecycle
import { Version, Author, Documented, Confidentiality } from meshr.annotations

@Version("1.0.0")
@Author(name="Data Team", email="data@company.com")
@Documented(summary="Customer entity with full governance")
@Confidentiality(level="confidential", classification="PII")
entity Customer with WithSecurity, WithStewardship, WithLifecycle is
  customer_id : String
  email : String
  personal_data : String

  aspects {
    DataClassification {
      level: "confidential",
```

```

        encryption_required: true
    },
    GDPRCompliance {
        data_subject_rights: List["access", "rectification", "erasure"],
        retention_period: Interval 7 year,
        lawful_basis: "consent",
        dpo_contact: "dpo@company.com"
    }
}
end

```

12.8.5 Roadmap

- ☒ **Phase 1** : Modules security et governance
 - ☒ **Phase 2** : Modules lifecycle , types et annotations
 - ☒ **Phase 3** : Exemples d'utilisation et documentation
 - ☐ **Phase 4** : Modules business et analytics
 - ☐ **Phase 5** : Intégrations réglementaires
 - ☐ **Phase 6** : Outils de validation et généralisation avancés
-

12.9 À venir

- Structuration des aspects (inline vs référence)
 - Système de types
 - Imports / Références croisées
 - Mécanisme de validation
 - Export YAML et JSON (Rego, Aspect (DataPlex Universal Catalog)...)
-

12.10 Exemples pratiques et cas d'usage

Cette section présente des exemples concrets d'utilisation de Meshr-Lang dans différents contextes métier et techniques.

12.10.1 Cas d'usage 1 : E-commerce et gestion des commandes

```
@Version("2.1.0")
@author(name="Product Team", email="product@ecommerce.com")
@Documented(summary="Product catalog management")
module ecommerce.products

import { DataClassification, WithSecurity } from meshr.security
import { DataStewardship, WithStewardship } from meshr.governance
import { WithLifecycle, WithVersioning } from meshr.lifecycle

export { Product, ProductCategory, ProductVariant }

// ===== ANUMÉRATIONS =====

enum ProductStatus is ("draft", "active", "discontinued", "archived")
enum CategoryType is ("physical", "digital", "service", "subscription")

// ===== TRAITS RÉUTILISABLES =====

trait WithPricing is
  base_price : Float
  currency : String
  tax_rate : Float
end

trait WithInventory is
  stock_quantity : Integer
  min_stock_level : Integer
  max_stock_level : Integer
end

// ===== ENTITÉS =====
```

```

entity Product with WithSecurity , WithStewardship , WithLifecycle , WithVersioning , V
  product_id : String
  name : String length 1..200
  description : String length 0..2000
  sku : String pattern "[A-Z0-9-]+$"
  status : ProductStatus
  category_id : String
  weight : Float
  dimensions : String

  aspects {
    DataClassification {
      level: "internal",
      encryption_required: false ,
      access_control: "role-based",
      data_retention: Interval 10 year ,
      steward: "product@ecommerce.com"
    },
    DataStewardship {
      steward: "product.team@ecommerce.com" ,
      owner: "business.team@ecommerce.com" ,
      business_domain: "product_catalog",
      last_review: "2024-01-15",
      next_review: "2024-07-15",
      contact_email: "product.team@ecommerce.com"
    }
  }
end

```

```

entity ProductCategory with WithSecurity , WithStewardship is
  category_id : String
  name : String length 1..100
  description : String length 0..500
  parent_category_id : String
  category_type : CategoryType
  display_order : Integer

  aspects {
    DataClassification {

```

```

        level: "public",
        encryption_required: false ,
        access_control: "public",
        data_retention: Interval 15 year ,
        steward: "product@ecommerce.com"
    }
}
end

// ===== RELATIONS =====

relation ProductBelongsToCategory is
    from Product(category_id)
    to ProductCategory(category_id)
end

relation CategoryHierarchy is
    from ProductCategory(category_id)
    to ProductCategory(parent_category_id)
end

@Version("1.5.0")
@author(name="Order Team", email="orders@ecommerce.com")
@Documented(summary="Order management system")
module ecommerce.orders

import { Product, ProductCategory } from ecommerce.products
import { DataClassification, GDPRCompliance, WithSecurity, WithGDPR } from meshr.se
import { DataStewardship, WithStewardship } from meshr.governance

export { Order, OrderItem, Customer, OrderStatus }

// ===== ANUMĂRATIONS =====

enum OrderStatus is ("pending", "confirmed", "processing", "shipped", "delivered",
enum PaymentStatus is ("pending", "paid", "failed", "refunded")
enum ShippingMethod is ("standard", "express", "overnight", "pickup")

```

```
// ===== TRAITS =====
```

```
trait WithAudit is
  created_at : Timestamp
  updated_at : Timestamp
  created_by : String
end
```

```
trait WithAddress is
  street : String
  city : String
  postal_code : String
  country : String
end
```

```
// ===== ENTITÃS =====
```

```
entity Customer with WithSecurity , WithGDPR, WithStewardship , WithAudit is
  customer_id : String
  email : String pattern "^[^@]+@[^@]+$"
  first_name : String length 1..50
  last_name : String length 1..50
  phone : String length 10..15
  birth_date : Date
  registration_date : Date
```

```
aspects {
  DataClassification {
    level: "confidential",
    encryption_required: true ,
    access_control: "role-based",
    data_retention: Interval 7 year ,
    steward: "privacy@ecommerce.com"
  },
  GDPRCompliance {
    data_subject_rights: List["access", "rectification", "erasure", "portability"
    retention_period: Interval 7 year ,
    lawful_basis: "consent",
    dpo_contact: "dpo@ecommerce.com",
    consent_required: true ,
```

```

        anonymization_required: true
    }
}
end

```

entity Order with WithSecurity, WithStewardship, WithAudit is

```

    order_id : String
    customer_id : String
    order_date : Date
    status : OrderStatus
    payment_status : PaymentStatus
    total_amount : Float
    currency : String
    shipping_method : ShippingMethod
    shipping_address : String
    billing_address : String
    notes : String length 0..1000

```

```

aspects {
    DataClassification {
        level: "confidential",
        encryption_required: true,
        access_control: "role-based",
        data_retention: Interval 7 year,
        steward: "orders@ecommerce.com"
    }
}

```

end

entity OrderItem with WithAudit is

```

    order_item_id : String
    order_id : String
    product_id : String
    quantity : Integer
    unit_price : Float
    total_price : Float

```

end

// ===== RELATIONS =====

```

relation CustomerPlacesOrder is
  from Customer(customer_id)
  to Order(customer_id)
end

relation OrderContainsItem is
  from Order(order_id)
  to OrderItem(order_id)
end

relation OrderItemReferencesProduct is
  from OrderItem(product_id)
  to Product(product_id)
end

```

12.10.2 Cas d'usage 2 : Architecture Data Mesh - Domaine RH

```

@Version("3.0.0")
@author(name="HR Data Team", email="hr.data@company.com")
@Documented(summary="Human Resources data domain")
module hr.employees

import { DataClassification, WithSecurity } from meshr.security
import { DataStewardship, WithStewardship, WithQuality } from meshr.governance
import { WithLifecycle, WithVersioning } from meshr.lifecycle

export { Employee, Department, Position, EmployeeSkill }

// ===== ANUMÉRATIONS =====

enum EmploymentStatus is ("active", "inactive", "terminated", "on_leave")
enum EmploymentType is ("full_time", "part_time", "contractor", "intern")
enum SkillLevel is ("beginner", "intermediate", "advanced", "expert")

// ===== TRAITS =====

trait WithPersonalInfo is
  first_name : String length 1..50

```

```

    last_name : String length 1..50
    email : String pattern "^[^@]+@[^@]+$"
    phone : String length 10..15
end

```

```

trait WithEmploymentInfo is
  employee_id : String
  hire_date : Date
  employment_type : EmploymentType
  employment_status : EmploymentStatus
  salary : Float
  currency : String
end

```

```

// ===== ENTITÃS =====

```

```

entity Employee with WithSecurity , WithStewardship , WithQuality , WithLifecycle , With
  department_id : String
  position_id : String
  manager_id : String
  office_location : String
  work_schedule : String

  aspects {
    DataClassification {
      level: "confidential",
      encryption_required: true ,
      access_control: "role-based",
      data_retention: Interval 10 year ,
      steward: "hr@company.com"
    },
    DataStewardship {
      steward: "hr.data@company.com",
      owner: "hr.team@company.com",
      business_domain: "human_resources",
      last_review: "2024-01-01",
      next_review: "2024-12-31",
      contact_email: "hr.data@company.com"
    }
  }
}

```

end

entity Department with WithSecurity , WithStewardship is

department_id : String

name : String length 1..100

description : String length 0..500

budget : Float

head_of_department : String

aspects {

 DataClassification {

 level: "internal",

 encryption_required: false ,

 access_control: "role-based",

 data_retention: Interval 15 year ,

 steward: "hr@company.com"

 }

}

end

// ===== RELATIONS =====

relation EmployeeBelongsToDepartment is

 from Employee(department_id)

 to Department(department_id)

end

relation EmployeeReportsTo is

 from Employee(employee_id)

 to Employee(manager_id)

end

12.10.3 Cas d'usage 3 : Secteur financier - Gestion des comptes

@Version("1.8.0")

@Author(name="Banking Data Team", email="banking.data@bank.com")

@Documented(summary="Banking account management with compliance")

module banking.accounts

```

import { DataClassification , WithSecurity } from meshr.security
import { DataStewardship , WithStewardship } from meshr.governance
import { WithLifecycle , WithVersioning } from meshr.lifecycle

export { Account , Transaction , Customer , AccountType }

// ===== ÃNUMÃRATIONS =====

enum AccountType is ("checking", "savings", "credit", "investment", "business")
enum TransactionType is ("deposit", "withdrawal", "transfer", "payment", "fee")
enum AccountStatus is ("active", "suspended", "closed", "frozen")

// ===== TRAITS =====

trait WithFinancialInfo is
  balance : Float
  currency : String
  interest_rate : Float
end

trait WithCompliance is
  kyc_status : String
  aml_risk_level : String
  last_kyc_review : Date
end

// ===== ENTITÃS =====

entity Customer with WithSecurity , WithStewardship , WithCompliance is
  customer_id : String
  ssn : String pattern "[0-9]{3}-[0-9]{2}-[0-9]{4}$"
  first_name : String length 1..50
  last_name : String length 1..50
  email : String pattern "^[^@]+@[^@]+$"
  phone : String length 10..15
  address : String
  date_of_birth : Date

  aspects {

```

```

        DataClassification {
            level: "restricted",
            encryption_required: true,
            access_control: "role-based",
            data_retention: Interval 10 year,
            steward: "compliance@bank.com"
        }
    }
end

```

```

entity Account with WithSecurity, WithStewardship, WithLifecycle, WithVersioning, V
    account_id : String
    customer_id : String
    account_number : String pattern "[0-9]{10}$"
    account_type : AccountType
    status : AccountStatus
    opened_date : Date
    closed_date : Date

    aspects {
        DataClassification {
            level: "restricted",
            encryption_required: true,
            access_control: "role-based",
            data_retention: Interval 10 year,
            steward: "banking@bank.com"
        }
    }
end

```

```

entity Transaction with WithSecurity, WithStewardship is
    transaction_id : String
    account_id : String
    transaction_type : TransactionType
    amount : Float
    currency : String
    transaction_date : Timestamp
    description : String length 0..200
    reference_number : String

```

```

aspects {
  DataClassification {
    level: "restricted",
    encryption_required: true,
    access_control: "role-based",
    data_retention: Interval 7 year,
    steward: "banking@bank.com"
  }
}
end

```

```
// ===== RELATIONS =====
```

```

relation CustomerOwnsAccount is
  from Customer(customer_id)
  to Account(customer_id)
end

```

```

relation AccountHasTransaction is
  from Account(account_id)
  to Transaction(account_id)
end

```

12.10.4 Cas d'usage 4 : Intégration technique - APIs et microservices

```

@Version("1.2.0")
@author(name="API Team", email="api@company.com")
@Documented(summary="API management and integration")
module apis.management

import { DataClassification, WithSecurity } from meshr.security
import { DataStewardship, WithStewardship } from meshr.governance
import { WithLifecycle, WithVersioning } from meshr.lifecycle

export { API, Endpoint, Service, Integration }

// ===== ANNOTATIONS =====

```

```

enum APIType is ("rest", "graphql", "grpc", "soap", "webhook")
enum ServiceStatus is ("active", "maintenance", "deprecated", "retired")
enum AuthenticationType is ("api_key", "oauth2", "jwt", "basic", "none")

```

```
// ===== TRAITS =====
```

```

trait WithTechnicalInfo is
  version : String
  base_url : String
  documentation_url : String
  repository_url : String
end

```

```

trait WithMonitoring is
  health_check_url : String
  metrics_endpoint : String
  alerting_enabled : Boolean
end

```

```
// ===== ENTITIES =====
```

```

entity API with WithSecurity , WithStewardship , WithLifecycle , WithVersioning , WithTechnicalInfo , WithMonitoring {
  api_id : String
  name : String length 1..100
  description : String length 0..500
  api_type : APIType
  status : ServiceStatus
  authentication_type : AuthenticationType
  rate_limit : Integer
  timeout_ms : Integer

  aspects {
    DataClassification {
      level: "internal",
      encryption_required: false ,
      access_control: "role-based",
      data_retention: Interval 5 year ,
      steward: "api@company.com"
    }
  }
}

```

end

entity Endpoint with WithSecurity, WithStewardship is

```
    endpoint_id : String
    api_id : String
    path : String
    method : String
    description : String length 0..300
    response_schema : String
    request_schema : String
```

```
    aspects {
        DataClassification {
            level: "internal",
            encryption_required: false,
            access_control: "role-based",
            data_retention: Interval 5 year,
            steward: "api@company.com"
        }
    }
```

end

entity Service with WithSecurity, WithStewardship, WithLifecycle, WithVersioning is

```
    service_id : String
    name : String length 1..100
    description : String length 0..500
    status : ServiceStatus
    deployment_environment : String
    container_image : String
    resource_requirements : String
```

```
    aspects {
        DataClassification {
            level: "internal",
            encryption_required: false,
            access_control: "role-based",
            data_retention: Interval 3 year,
            steward: "devops@company.com"
        }
    }
```

}

```

end

// ===== RELATIONS =====

relation APIHasEndpoint is
  from API(api_id)
  to Endpoint(api_id)
end

relation ServiceExposesAPI is
  from Service(service_id)
  to API(api_id)
end

```

12.11 Bonnes pratiques et conventions

Cette section présente les bonnes pratiques recommandées pour écrire du code Meshr-lang efficace, maintenable et conforme aux standards de l'organisation.

12.11.1 Conventions de nommage

12.11.1.1 Modules

- **Format** : domain.subdomain (ex: marketing.analytics, core.types)
- **Convention** : Utiliser des noms en minuscules avec des points comme séparateurs
- **À éviter** : Les noms trop génériques comme common, utils, shared

12.11.1.2 Entités et types

- **Format** : PascalCase (ex: Customer, ProductCatalog, OrderStatus)
- **Convention** : Noms explicites qui reflètent le concept métier
- **À éviter** : Abréviations et acronymes non standardisés

12.11.1.3 Champs et propriétés

- **Format** : camelCase (ex: customerId, emailAddress, createdAt)
- **Convention** : Noms descriptifs et cohérents
- **À éviter** : Noms génériques comme data, info, value

12.11.1.4 Numérations

- **Valeurs** : UPPER_SNAKE_CASE (ex: ACTIVE, PENDING_APPROVAL, HIGH_PRIORITY)
- **Convention** : Valeurs explicites et auto-documentées
- **Éviter** : Valeurs numériques ou abréviations

12.11.2 Organisation des modules

modules /

core /

- types.meshr # Types fondamentaux
- governance.meshr # Aspects de gouvernance
- security.meshr # Aspects de sécurité

business /

- customer.meshr # Domaine client
- product.meshr # Domaine produit
- order.meshr # Domaine commande

shared /

- common.meshr # Types partagés
- policies.meshr # Politiques transversales

12.11.2.2 Principes d'organisation

- **Un module = un domaine métier** : Éviter les modules trop larges
- **Dépendances claires** : Minimiser les dépendances circulaires
- **Séparation des responsabilités** : Séparer les types, aspects, et politiques

12.11.3 Utilisation des traits et aspects

12.11.3.1 Traits

- **Factorisation** : Créer des traits pour les patterns récurrents
- **Composition** : Préférer la composition à l'héritage complexe
- **Naming** : Préfixer par With (ex: WithAudit, WithTimestamps)

12.11.3.2 Aspects

- **Spécificité** : Créer des aspects spécialisés plutôt que génériques
- **Réutilisabilité** : Concevoir pour la réutilisation entre domaines
- **Documentation** : Toujours documenter le rôle et l'usage des aspects

12.11.4 Documentation et commentaires

12.11.4.1 Commentaires obligatoires

- **Modules** : Description du domaine et des responsabilités
- **Entités complexes** : Explication du rôle métier
- **Aspects personnalisés** : Usage et contraintes
- **Relations** : Cardinalité et rôles métier

/**

* Module de gestion des clients et de leurs données personnelles.

*

* Ce module définit les entités centrales du domaine client,

* incluant la conformité RGPD et les aspects de sécurité.

*/

module business.customer

// Entité représentant un client dans le système

entity Customer with WithAudit, WithSecurity is

 customerId: String pattern "^CUST-[0-9]{8}\$"

 email: String pattern "^[^@]+@[^@]+\$"

 // ... autres champs

end

12.11.5 Gestion des erreurs et validation

12.11.5.1 Contraintes de validation

- **Patterns** : Utiliser des expressions régulières pour valider les formats
- **Ranges** : Définir des plages de valeurs pour les nombres
- **Required fields** : Marquer explicitement les champs obligatoires

12.11.5.2 Gestion des versions

- **Versioning sémantique** : Suivre le format MAJOR.MINOR.PATCH
- **Rétrocompatibilité** : Éviter les breaking changes dans les versions mineures
- **Dépréciation** : Utiliser les annotations @deprecated pour les évolutions

12.11.6 S curit  et conformit 

12.11.6.1 Donn es sensibles

- **Classification** : Toujours classifier les donn es avec des aspects appropri s
- **Chiffrement** : Sp cifier les exigences de chiffrement
- **Acc s** : D finir les niveaux d'acc s et permissions

12.11.6.2 Conformit  r glementaire

- **RGPD** : Inclure les aspects de r tention et de consentement
- **Audit** : Tra abilit  des modifications et acc s
- **Gouvernance** : D finir les r les et responsabilit s

12.11.7 Tests et validation

12.11.7.1 Tests de mod les

- **Validation syntaxique** : V rifier la syntaxe des fichiers
- **Validation s mantique** : Tester les contraintes et relations
- **Tests d'int gration** : Valider les imports et d pendances

12.11.7.2 Outils recommand s

- **Linting** : Utiliser les outils de validation automatique
- **CI/CD** : Int grer la validation dans les pipelines
- **Documentation** : G n rer automatiquement la documentation

12.12 Annexes

12.12.1 Annexe A Grammaire EBNF (r f rence)

```
(* =====  
Meshr-Lang Grammaire EBNF (align e ANTLR)  
Mise   jour: 2025-09-17 - Version 0.2.0 avec m triques  
===== *)
```

```
compilation-unit    = module-decl , { import-decl }, { export-decl }, { top-level-decl } ;
```

```
module-decl         = { annotation }, "module", qualified-name ;
```

```
import-decl         = "import", import-items , "from", qualified-name ;
```

```

import-items      = "*"
                  | identifier
                  | "{", import-item, { ",", import-item }, "}" ;
import-item       = identifier ;

export-decl       = "export", ( export-items | exportable-decl ) ;
export-items      = "{", identifier, { ",", identifier }, "}" ;

top-level-decl    = enum-decl | annotation-decl | record-decl | trait-decl | aspect-decl ;
exportable-decl   = enum-decl | entity-decl | type-relation-decl | relation-decl ;

enum-decl         = { annotation }, [ "sealed" ], "enum", identifier, [ enum-signature ], enum-values ;
enum-signature    = "(", enum-attribute, { ",", enum-attribute }, ")" ;
enum-attribute    = identifier, ":", ( qualified-name | base-type ), [ "=", annotation-value ] ;
enum-values       = enum-value, { ",", enum-value } ;
enum-value        = ( identifier | string-literal ), [ "(", enum-value-args, ")" ] ;
enum-value-args   = enum-value-arg, { ",", enum-value-arg } ;
enum-value-arg    = identifier, "=", annotation-value ;

annotation-decl   = { annotation }, "annotation", identifier, "is", annotation-field ;
annotation-field  = ( "required" | "optional" ), identifier, ":", ( qualified-name | annotation ) ;

annotation        = "@", identifier, [ "(", [ annotation-args ], ")" ] ;
annotation-args   = annotation-arg, { ",", annotation-arg } ;
annotation-arg    = annotation-arg-pair | annotation-value ;
annotation-arg-pair = identifier, "=", annotation-value ;
annotation-value  = string-literal | number-literal | boolean-literal | qualified-name ;

base-type         = "Boolean" | string-type | "Integer" | "Float" | "Double"
                  | "Date" | "Datetime" | "Time" | "Timestamp"
                  | "Geography" | "Bytes" | "Json" | "Sql"
                  | "Interval" | "Range" ;

(* Type String avec contraintes optionnelles *)
string-type       = "String", [ string-constraints ] ;
string-constraints = string-constraint, { string-constraint } ;
string-constraint = "pattern", string-literal
                  | "length", signed-number, "..", signed-number ;

interval-literal  = "Interval", interval-single-part

```

```

| "Interval", string-literal , "year", "to", "month"
| "Interval", string-literal , "year", "to", "day"
| "Interval", string-literal , "year", "to", "hour"
| "Interval", string-literal , "year", "to", "minute"
| "Interval", string-literal , "year", "to", "second"
| "Interval", string-literal , "month", "to", "day"
| "Interval", string-literal , "month", "to", "hour"
| "Interval", string-literal , "month", "to", "minute"
| "Interval", string-literal , "month", "to", "second"
| "Interval", string-literal , "day", "to", "hour"
| "Interval", string-literal , "day", "to", "minute"
| "Interval", string-literal , "day", "to", "second"
| "Interval", string-literal , "hour", "to", "minute"
| "Interval", string-literal , "hour", "to", "second"
| "Interval", string-literal , "minute", "to", "second" ;

```

```

interval-single-part = [ '-' ], number-literal , interval-unit ;
interval-unit        = "year" | "quarter" | "month" | "week" | "day"
                      | "hour" | "minute" | "second" | "millisecond" | "microsecond"

```

```

qualified-name       = identifier , { ".", identifier } ;
boolean-literal      = "true" | "false" ;

```

(* ===== RECORD DECLARATION ===== *)

```

record-decl          = "record", identifier , [ with-clause ], "is", record-field , {
record-field          = identifier , ":", type-ref , [ "=", annotation-value ], [ NEWLINE

```

(* ===== TRAIT DECLARATION ===== *)

```

trait-decl           = [ "sealed" ], "trait", identifier , [ with-clause ], "is", tra
trait-field           = identifier , ":", type-ref , [ "=", annotation-value ], [ NEWLINE
with-clause           = "with", qualified-name, { ",", qualified-name } ;
trait-aspects         = "aspects", "{", aspect-instance, { ",", aspect-instance }, "}"

```

(* ===== ASPECT DECLARATION ===== *)

```

aspect-decl           = { annotation }, [ "abstract" ], "aspect", identifier , [ aspect
aspect-inheritance    = extends-clause , [ with-clause ]
                      | with-clause , [ extends-clause ] ;
extends-clause        = "extends", qualified-name ;
aspect-field          = identifier , ":", type-ref , [ "=", annotation-value ], [ NEWLINE

```

```

(* ===== ENTITY ===== *)
entity-decl      = { annotation }, [ "sealed" ], "entity", identifier , [ with-cl
entity-field     = identifier , ":", type-ref , [ "=", annotation-value ], [ NEWL
entity-aspects   = "aspects", "{", aspect-instance , { ",", aspect-instance }, "

(* ===== TYPE RELATION ===== *)
type-relation-decl = { annotation }, [ "sealed" ], "type", "relation", identifier ,
type-relation-field = identifier , ":", type-ref , [ "=", annotation-value ], [ NEWL
type-relation-aspects = "aspects", "{", aspect-instance , { ",", aspect-instance },

(* ===== RELATION ===== *)
relation-decl      = { annotation }, [ "sealed" ], [ "bidirectional" ], "relation"
relation-type      = "of", "type", qualified-name ;
relation-endpoints = "from", relation-endpoint , "to", relation-endpoint ;
relation-endpoint  = qualified-name , "(", identifier , { ",", identifier }, ")" ;
relation-field-list = relation-field , { relation-field } ;
relation-field     = identifier , ":", type-ref , [ "=", annotation-value ], [ NEWL
relation-aspects   = "aspects", "{", aspect-instance , { ",", aspect-instance }, "

(* ===== ASPECT INSTANCES ===== *)
aspect-instance     = identifier , "{", [ aspect-instance-field-list ], "}" ;
aspect-instance-field-list = aspect-instance-field , { ",", aspect-instance-field }
aspect-instance-field = identifier , ":", annotation-value ;

(* ===== COLLECTION/COMPOSITE LITTÉRAUX ===== *)
list-literal        = "List", "[", [ annotation-value , { ",", annotation-value } ],
map-literal         = "Map", "{", [ map-entry , { ",", map-entry } ], "}" ;
map-entry           = annotation-value , ":", annotation-value ;
record-literal      = "Record", "{", [ record-lit-field , { ",", record-lit-field } ],
record-lit-field    = identifier , ":", annotation-value ;
signed-number       = [ '-' ], number-literal ;
range-literal       = "Range", signed-number , "..", signed-number ;

(* ===== JSON LITTÉRAUX ===== *)
json-literal        = "Json", "{", json-object-content , "}"
                    | "Json", "[", json-array-content , "]"
                    | "Json", string-literal ;
json-object-content = json-pair , { ",", json-pair } ;
json-array-content  = json-value , { ",", json-value } ;
json-pair           = string-literal , ":", json-value ;

```

```

json-value          = string-literal
                    | number-literal
                    | boolean-literal
                    | "null"
                    | "{", json-object-content, "}"
                    | "[", json-array-content, "]" ;

```

```

(* ===== GEOGRAPHY LITTÉRAUX ===== *)
geography-literal   = "Geography", string-literal
                    | "Geography", "{", json-object-content, "}" ;

```

```

(* ===== BYTES LITTÉRAUX ===== *)
bytes-literal        = "Bytes", string-literal
                    | "Bytes", "[", bytes-array-content, "]"
                    | "Bytes", "{", json-object-content, "}" ;
bytes-array-content  = bytes-value, { ",", bytes-value } ;
bytes-value          = signed-number ;

```

```

(* ===== METRIC DECLARATION ===== *)
metric-decl          = { annotation }, [ "sealed" ], "metric", identifier, [ with-cl
                        metric-source,
                        ( metric-calculation | metric-aggregation ),
                        [ metric-unit ],
                        [ metric-outputs ],
                        [ metric-dimensions ],
                        [ metric-filters ],
                        [ metric-temporal ],
                        [ metric-aspects ],
                        "end" ;

```

```

metric-source        = "source", qualified-name ;
metric-calculation    = "calculation", expression ;
metric-aggregation    = "aggregation", "{", aggregation-field-list, "}" ;
aggregation-field-list = aggregation-field, { ",", aggregation-field } ;
aggregation-field     = identifier, "as", expression ;

```

```

metric-unit          = "unit", string-literal ;
metric-outputs        = "outputs", "{", output-field-list, "}" ;
output-field-list     = output-field, { ",", output-field } ;
output-field          = identifier, ":", type-ref ;

```

```

metric-dimensions    = "dimensions", "{", dimension-list, "}" ;
dimension-list       = dimension, { ",", dimension } ;
dimension            = identifier, ( expression | match-expression ) ;

match-expression     = "match", expression, "is", match-arms, "end" ;
match-arms           = match-arm, { ",", match-arm } ;
match-arm            = pattern, "->", match-result ;

pattern              = literal-pattern | range-pattern | enum-pattern | wildcard-pattern ;
literal-pattern      = string-literal | number-literal | boolean-literal ;
range-pattern        = number-literal, "..", [ number-literal ] ;
enum-pattern         = qualified-name ;
wildcard-pattern     = "_" ;
or-pattern           = pattern, { "or", pattern } ;
match-result         = string-literal | number-literal | boolean-literal | qualified-name ;

metric-filters       = "filters", "{", filter-list, "}" ;
filter-list          = filter-expression, { ",", filter-expression } ;
filter-expression    = expression ;

metric-temporal      = "temporal", "{", temporal-config-list, "}" ;
temporal-config-list = temporal-config, { ",", temporal-config } ;
temporal-config      = temporal-field, interval-literal ;
temporal-field       = "window" | "refresh_frequency" | "historical_depth" ;

metric-aspects       = "aspects", "{", aspect-instance-list, "}" ;

(* ===== EXPRESSIONS ===== *)
expression           = arithmetic-expr | comparison-expr | logical-expr | function-call ;

(* ===== TERMINAUX ===== *)
string-literal       = "'", { character - "'" | '\\'' }, "'" ;
number-literal       = digit, { digit } | "0x", hex-digit, { hex-digit } ;
identifier           = letter, { letter | digit | "_" } ;
NEWLINE              = ("\r"?), "\n", { ("\r"?), "\n" } ;

comment              = line-comment | block-comment ;
line-comment         = "//", { character - end-of-line } ;
block-comment        = "/*", { character - "*/" }, "*/" ;

```

12.12.2 Annexe B Grammaire ANTLR (rÃfÃrence, Python target)

Note: Cette annexe contient un extrait de la grammaire ANTLR. La **grammaire complÃte et Ã jour** (incluant les mÃtriques et le pattern matching) se trouve dans le fichier /grammar/MeshrModule.g4 du projet. DerniÃre mise Ã jour : **Version 0.2.0 (2025-09-17)** avec support complet des mÃtriques.

```
grammar MeshrModule ;
```

```
// =====
```

```
// EntrÃe principale
```

```
// =====
```

```
compilationUnit
```

```
    : annotatedModuleDecl importDecl* exportDecl* topLevelDecl* EOF
    ;
```

```
// =====
```

```
// DÃclarations de module
```

```
// =====
```

```
annotatedModuleDecl
```

```
    : annotation* 'module' qualifiedName
    ;
```

```
// =====
```

```
// Import / Export
```

```
// =====
```

```
importDecl
```

```
    : 'import' importItemsWithOptionalBraces 'from' qualifiedName
    ;
```

```
importItemsWithOptionalBraces
```

```
    : IDENTIFIER                                # SingleImport
    | '*'                                         # WildcardImport
    | '{' importItems '}'                       # GroupImport
    ;
```

```
importItems
```

```
    : IDENTIFIER (',' IDENTIFIER)+
    ;
```

```
exportDecl
```

```

        : 'export ' exportableDecl                # InlineExport
        | 'export ' '{' exportItems '}'          # GroupedExport
        ;

exportItems
    : IDENTIFIER (',' IDENTIFIER)*
    ;

// =====
// DÃclarations de haut niveau
// =====
topLevelDecl
    : annotatedEnumDecl
    | sealedEnumDecl
    | annotatedAnnotationDecl
    | sealedRecordDecl
    | recordDecl
    | sealedTraitDecl
    | traitDecl
    | annotatedAspectDecl
    | sealedAspectDecl
    | annotatedEntityDecl
    | sealedEntityDecl
    | annotatedTypeRelationDecl
    | sealedTypeRelationDecl
    | annotatedRelationDecl
    | sealedRelationDecl
    ;

// ===== ENUM =====
exportableDecl
    : enumDecl
    | sealedEnumDecl
    | entityDecl
    | sealedEntityDecl
    | typeRelationDecl
    | sealedTypeRelationDecl
    | relationDecl
    | sealedRelationDecl
    ;

```

```

annotatedEnumDecl
    : annotation* enumDecl
    ;

sealedEnumDecl
    : SEALED enumDecl
    ;

// ===== ENTITY =====
annotatedEntityDecl
    : annotation* entityDecl
    ;

sealedEntityDecl
    : SEALED entityDecl
    ;

enumDecl
    : 'enum' IDENTIFIER enumSignature? 'is' '(' enumValueList ')'
    ;

enumSignature
    : '(' enumAttributeList ')'
    ;

enumAttributeList
    : enumAttribute (',' enumAttribute)*
    ;

enumAttribute
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;

enumValueList
    : enumValue (',' enumValue)*
    ;

enumValue
    : (IDENTIFIER | STRING_LITERAL) '(' enumValueArgList? ')'

```

```

;

enumValueArgList
    : enumValueArg (',' enumValueArg)*
    ;

enumValueArg
    : IDENTIFIER '=' annotationValue
    ;

// ===== ENTITY DECLARATION =====
entityDecl
    : ENTITY IDENTIFIER withClause? 'is' entityFieldList entityAspects? 'end'
    ;

entityFieldList
    : entityField+
    ;

entityField
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;

entityAspects
    : ASPECTS '{' aspectInstanceList '}'
    ;

aspectInstanceList
    : aspectInstance (',' aspectInstance)*
    ;

aspectInstance
    : IDENTIFIER '{' aspectInstanceFieldList? '}'
    ;

aspectInstanceFieldList
    : aspectInstanceField (',' aspectInstanceField)*
    ;

aspectInstanceField

```

```

        : IDENTIFIER ':' annotationValue
        ;

// ===== TYPE RELATION DECLARATION =====
annotatedTypeRelationDecl
    : annotation* typeRelationDecl
    ;

sealedTypeRelationDecl
    : SEALED typeRelationDecl
    ;

typeRelationDecl
    : TYPE RELATION IDENTIFIER withClause? 'is ' typeRelationFieldList typeRelationA
    ;

typeRelationFieldList
    : typeRelationField+
    ;

typeRelationField
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;

typeRelationAspects
    : ASPECTS '{' aspectInstanceList '}'
    ;

// ===== RELATION DECLARATION =====
annotatedRelationDecl
    : annotation* relationDecl
    ;

sealedRelationDecl
    : SEALED relationDecl
    ;

relationDecl
    : BIDIRECTIONAL? RELATION IDENTIFIER relationType? withClause? 'is ' relationEnd
    ;

```

```
relationType
    : 'of' TYPE IDENTIFIER
    ;
```

```
relationEndpoints
    : FROM relationEndpoint TO relationEndpoint
    ;
```

```
relationEndpoint
    : IDENTIFIER '(' IDENTIFIER (',' IDENTIFIER)* ')'
    ;
```

```
relationFieldList
    : relationField+
    ;
```

```
relationField
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;
```

```
relationAspects
    : ASPECTS '{' aspectInstanceList '}'
    ;
```

```
// ===== ANNOTATION DECLARATION =====
```

```
annotatedAnnotationDecl
    : annotation* annotationDecl
    ;
```

```
annotationDecl
    : 'annotation' IDENTIFIER 'is' annotationFieldList 'end'
    ;
```

```
annotationFieldList
    : annotationField+
    ;
```

```
annotationField
    : ('required' | 'optional') IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE
```

```

;

// ===== ANNOTATION USAGE =====
annotation
    : '@' IDENTIFIER ('(' annotationArgs? ')')?
    ;

annotationArgs
    : annotationArg (',' annotationArg)*
    ;

annotationArg
    : annotationArgPair
    | annotationValue
    ;

annotationArgPair
    : IDENTIFIER '=' annotationValue
    ;

// ===== TYPES DE BASE =====

// Types de base reconnus
baseType
    : 'Boolean '
    | stringType
    | 'Integer '
    | 'Float '
    | 'Double '
    | 'Date '
    | 'Datetime '
    | 'Time '
    | 'Geography '
    | 'Timestamp '
    | 'Bytes '
    | 'Json '
    | 'Sql '
    | 'Interval '
    | 'Range '
    ;

```

```

// Type String avec contraintes optionnelles
stringType
  : 'String ' stringConstraints?
  ;

stringConstraints
  : stringConstraint (stringConstraint)*
  ;

stringConstraint
  : 'pattern ' STRING_LITERAL
  | 'length ' signedNumber '..' signedNumber
  ;

// ===== SHARED =====
qualifiedName
  : IDENTIFIER ( '.' IDENTIFIER)*
  ;

// ===== TYPE REFERENCES =====
typeRef
  : qualifiedName
  | baseType
  | listType
  | mapType
  | rangeType
  ;

listType
  : 'List ' 'of' typeRef
  ;

mapType
  : 'Map' 'of' typeRef 'to' typeRef
  ;

rangeType
  : 'Range' 'of' typeRef
  ;

```

```
// ===== ANNOTATION USAGE =====
```

```
annotationValue
```

```
  : STRING_LITERAL
  | NUMBER_LITERAL
  | BOOLEAN_LITERAL
  | qualifiedName
  | intervalLiteral
  | listLiteral
  | mapLiteral
  | recordLiteral
  | rangeLiteral
  | jsonLiteral
  | geographyLiteral
  | bytesLiteral
  | datetimeLiteral
  | dateLiteral
  | timeLiteral
  | timestampLiteral
  | sqlLiteral
  ;
```

```
// ===== INTERVAL LITERALS =====
```

```
intervalLiteral
```

```
  : 'Interval' intervalSinglePart
  | 'Interval' STRING_LITERAL 'year' 'to' 'month'
  | 'Interval' STRING_LITERAL 'year' 'to' 'day'
  | 'Interval' STRING_LITERAL 'year' 'to' 'hour'
  | 'Interval' STRING_LITERAL 'year' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'year' 'to' 'second'
  | 'Interval' STRING_LITERAL 'month' 'to' 'day'
  | 'Interval' STRING_LITERAL 'month' 'to' 'hour'
  | 'Interval' STRING_LITERAL 'month' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'month' 'to' 'second'
  | 'Interval' STRING_LITERAL 'day' 'to' 'hour'
  | 'Interval' STRING_LITERAL 'day' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'day' 'to' 'second'
  | 'Interval' STRING_LITERAL 'hour' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'hour' 'to' 'second'
  | 'Interval' STRING_LITERAL 'minute' 'to' 'second'
```

```

;

intervalSinglePart
    : '-'? NUMBER_LITERAL intervalUnit
    ;

intervalUnit
    : 'year' | 'quarter' | 'month' | 'week' | 'day'
    | 'hour' | 'minute' | 'second' | 'millisecond' | 'microsecond'
    ;

// ===== RECORD DECLARATION =====
recordDecl
    : 'record' IDENTIFIER withClause? 'is' recordFieldList 'end'
    ;

sealedRecordDecl
    : SEALED recordDecl
    ;

recordFieldList
    : recordField+
    ;

recordField
    : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
    ;

// ===== TRAIT DECLARATION =====
traitDecl
    : 'trait' IDENTIFIER withClause? 'is' traitFieldList traitAspects? 'end'
    ;

sealedTraitDecl
    : SEALED traitDecl
    ;

traitFieldList
    : traitField+
    ;

```

```

traitField
  : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
  ;

traitAspects
  : ASPECTS '{' aspectInstanceList '}'
  ;

withClause
  : 'with' qualifiedName (',' qualifiedName)*
  ;

// ===== ASPECT DECLARATION =====
annotatedAspectDecl
  : annotation* aspectDecl
  ;

aspectDecl
  : 'abstract'? 'aspect' IDENTIFIER aspectInheritance? 'is' aspectFieldList 'end'
  ;

sealedAspectDecl
  : SEALED aspectDecl
  ;

aspectInheritance
  : extendsClause withClause?
  | withClause extendsClause?
  ;

extendsClause
  : 'extends' qualifiedName
  ;

aspectFieldList
  : aspectField+
  ;

aspectField

```

```

        : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
        ;

// ===== COLLECTION AND COMPOSITE LITERALS =====
listLiteral
    : 'List' '[' (annotationValue (',' annotationValue)*)? ']'
    ;

mapLiteral
    : 'Map' '{' (mapEntry (',' mapEntry)*)? '}'
    ;

mapEntry
    : annotationValue ':' annotationValue
    ;

recordLiteral
    : 'Record' '{' (recordLitField (',' recordLitField)*)? '}'
    ;

recordLitField
    : IDENTIFIER ':' annotationValue
    ;

rangeLiteral
    : 'Range' signedNumber '..' signedNumber
    ;

// ===== JSON LITERALS =====
jsonLiteral
    : 'Json' '{' jsonObjectContent '}'
    | 'Json' '[' jsonArrayContent ']'
    | 'Json' STRING_LITERAL
    ;

jsonObjectContent
    : jsonPair (',' jsonPair)*
    | // empty
    ;

```

```

jsonArrayContent
    : jsonValue (',' jsonValue)*
    | // empty
    ;

jsonPair
    : STRING_LITERAL ':' jsonValue
    ;

jsonValue
    : STRING_LITERAL
    | NUMBER_LITERAL
    | BOOLEAN_LITERAL
    | 'null'
    | '{' jsonObjectContent '}'
    | '[' jsonArrayContent ']'
    ;

signedNumber
    : '-'? NUMBER_LITERAL
    ;

// ===== GEOGRAPHY LITERALS =====
geographyLiteral
    : 'Geography' STRING_LITERAL
    | 'Geography' '{' jsonObjectContent '}'
    ;

// ===== BYTES LITERALS =====
bytesLiteral
    : 'Bytes' STRING_LITERAL
    | 'Bytes' '[' byteArrayContent ']'
    | 'Bytes' '{' jsonObjectContent '}'
    ;

byteArrayContent
    : bytesValue (',' bytesValue)*
    | // empty
    ;

```

```

bytesValue
    : signedNumber
    ;

// ===== TEMPORAL LITERALS =====
datetimeLiteral
    : 'Datetime' STRING_LITERAL
    ;

dateLiteral
    : 'Date' STRING_LITERAL
    ;

timeLiteral
    : 'Time' STRING_LITERAL
    ;

timestampLiteral
    : 'Timestamp' STRING_LITERAL
    ;

// ===== SQL LITERALS =====
sqlLiteral
    : 'Sql' STRING_LITERAL
    ;

// ===== TERMINALS =====
fragment ESC
    : '\\' ["\\ / bfnrt]
    ;

AT: '@';
STRING_LITERAL: "'" (ESC | ~["\\ r\n])* "'";

// Keywords
SEALED: 'sealed';
ENTITY: 'entity';
TYPE: 'type';
RELATION: 'relation';
BIDIRECTIONAL: 'bidirectional';

```

```

FROM: 'from ';
TO: 'to ';
ASPECTS: 'aspects ';

NUMBER_LITERAL
    : [0-9]+ ( '.' [0-9]+ )?
    | '0' [xX] [0-9a-fA-F]+
    ;

BOOLEAN_LITERAL
    : 'true ' | 'false '
    ;

IDENTIFIER: [a-zA-Z_][a-zA-Z0-9_]* ;

WS: [ \t\r\n]+ -> skip ;
NEWLINE: ( '\r'? '\n')+ -> skip ;
COMMENT: '//' ~[\r\n]* -> skip ;
MULTILINE_COMMENT: '/*' .*? '*/' -> skip ;

```

12.13 Annexe C : Guide complet des littéraux

Cette annexe présente tous les types de littéraux supportés par Meshr-lang avec des exemples concrets et pratiques.

12.13.1 Littéraux primitifs

```

string_literal : String = "Hello World"
empty_string  : String = ""
quoted_string : String = "Il a dit \"Bonjour\""

```

```

decimal_literal : Integer = 42
hex_literal     : Integer = 0xFF0000
octal_literal   : Integer = 0o755
binary_literal  : Integer = 0b1010
negative_literal : Integer = -100

```

```
float_literal : Float = 3.14
scientific_literal : Float = 1.23e-4
double_literal : Double = 3.14159265359
```

```
true_literal : Boolean = true
false_literal : Boolean = false
```

12.13.2 Lists and composites

```
string_list : List of String = List["item1", "item2", "item3"]
number_list : List of Integer = List[1, 2, 3, 4, 5]
mixed_list : List of String = List["alpha", "beta", "gamma"]
empty_list : List of String = List[]
```

```
simple_map : Map of String to Integer = Map{"key1": 1, "key2": 2}
nested_map : Map of String to String = Map{"user": "admin", "role": "superuser"}
empty_map : Map of String to Integer = Map{}
```

```
user_record : Record = Record{name: "Alice", age: 42, active: true}
config_record : Record = Record{host: "localhost", port: 8080, ssl: true}
```

```
integer_range : Range of Integer = Range 1..100
negative_range : Range of Integer = Range -50..50
float_range : Range of Float = Range 0.0..1.0
```

12.13.3 Specialized lists

```

// Format WKT (Well-Known Text)
point_wkt : Geography = Geography"POINT(2.3522 48.8566)"
polygon_wkt : Geography = Geography"POLYGON((0 0, 1 0, 1 1, 0 1, 0 0))"
linestring_wkt : Geography = Geography"LINESTRING(0 0, 1 1, 2 2)"

// Format GeoJSON
point_geojson : Geography = Geography{"type":"Point","coordinates":[2.3522,48.8566]}
polygon_geojson : Geography = Geography{"type":"Polygon","coordinates": [[[0,0],[1,0],[1,1],[0,1],[0,0]]]}

// Objet JSON
json_object : Json = Json{"name": "test", "value": 123, "active": true}
json_array : Json = Json[1, 2, 3, "hello", true]
json_nested : Json = Json{"user": {"name": "Alice", "preferences": {"theme": "dark"}}}

// JSON en chaîne brute
json_string : Json = Json{"name": "test", "value": 123}

// Tableau d'octets
byte_array : Bytes = Bytes[0, 1, 2, 3, 255]
hex_bytes : Bytes = Bytes[0xFF, 0x00, 0xFF, 0x00]

// Chaîne hexadécimale
hex_string : Bytes = Bytes"deadbeef"
base64_string : Bytes = Bytes"SGVsbG8gV29ybGQ="

// Objet JSON avec métadonnées
json_bytes : Bytes = Bytes{"data": "U29tZSBkYXRh", "encoding": "base64", "size": 20}

```

12.13.4 ° Littéraux temporels

```

// Date seule (format ISO 8601)
date_literal : Date = Date"2021-01-01"
date_with_timezone : Date = Date"2024-12-31"

```

```

// Heure seule
time_literal : Time = Time"00:00:00"
time_with_millis : Time = Time"14:30:25.123"
time_with_timezone : Time = Time"14:30:25Z"

// Date et heure compl te
datetime_literal : Datetime = Datetime"2021-01-01T00:00:00Z"
datetime_with_millis : Datetime = Datetime"2024-12-31T23:59:59.999Z"
datetime_local : Datetime = Datetime"2024-03-15T14:30:25"

// Timestamp ( quivalent   Datetime)
timestamp_literal : Timestamp = Timestamp"2021-01-01T00:00:00Z"
timestamp_millis : Timestamp = Timestamp"2024-01-01T00:00:00.123Z"

// Intervalles simples
year_interval : Interval = Interval 1 year
month_interval : Interval = Interval 6 months
day_interval : Interval = Interval 30 days
hour_interval : Interval = Interval 2 hours
minute_interval : Interval = Interval 45 minutes
second_interval : Interval = Interval 30 seconds

// Intervalles n gatifs
negative_interval : Interval = Interval -1 day
past_interval : Interval = Interval -2 hours

// Intervalles complexes
complex_interval : Interval = Interval 1 year 6 months 15 days

```

12.13.5 Litt raux SQL

```

// Requ tes simples
select_query : Sql = Sql"SELECT * FROM users WHERE active = true"
insert_query : Sql = Sql"INSERT INTO logs (message, timestamp) VALUES (?, ?)"
update_query : Sql = Sql"UPDATE products SET price = ? WHERE id = ?"
delete_query : Sql = Sql"DELETE FROM table WHERE id = ?"

```

```

// Requêtes complexes
join_query : Sql = Sql"SELECT u.id, u.name, p.title FROM users u JOIN products p ON
subquery : Sql = Sql"SELECT * FROM (SELECT id, name FROM users WHERE created_at > '
cte_query : Sql = Sql"WITH RECURSIVE category_tree AS (SELECT id, name, parent_id F

// Requêtes avec fonctions
aggregate_query : Sql = Sql"SELECT COUNT(*), AVG(price), MAX(created_at) FROM produ
window_function : Sql = Sql"SELECT id, name, ROW_NUMBER() OVER (PARTITION BY categ
case_statement : Sql = Sql"SELECT id, name, CASE WHEN price > 100 THEN 'expensive'

// DDL (Data Definition Language)
create_table : Sql = Sql"CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(255),
create_index : Sql = Sql"CREATE INDEX idx_users_email ON users(email)"
create_view : Sql = Sql"CREATE VIEW active_users AS SELECT * FROM users WHERE activ

```

12.13.6 Exemples d'usage dans les annotations

```

@ApiConfig(
    endpoint="https://api.example.com",
    headers=Json{"Content-Type": "application/json"},
    timeout=Interval 30 seconds,
    retry_count=3
)
annotation ApiConfig is
    required endpoint : String
    optional headers : Json = Json{"Content-Type": "application/json"}
    optional timeout : Interval = Interval 30 seconds
    optional retry_count : Integer = 3
end

@LocationConfig(
    office=Geography"POINT(2.3522 48.8566)",
    coverage_area=Geography"POLYGON((2.3522 48.8566, 2.3523 48.8567, 2.3524 48.8568
    business_hours=Time"09:00:00" to Time"17:00:00"
)
annotation LocationConfig is
    required office : Geography
    optional coverage_area : Geography = Geography"POINT(0.0 0.0)"
    optional business_hours : Range of Time = Range Time"09:00:00"..Time"17:00:00"

```

end

```
@AuditConfig(  
    created=Datetime"2024-01-01T00:00:00Z",  
    retention=Interval 7 years,  
    encryption_key=Bytes"deadbeef",  
    query=Sql"SELECT * FROM audit_logs WHERE created_at >= ?"  
)  
annotation AuditConfig is  
    required created : Datetime  
    optional retention : Interval = Interval 1 year  
    optional encryption_key : Bytes = Bytes""  
    optional query : Sql = Sql"SELECT * FROM audit_logs"  
end
```

12.13.7 Exemples d'usage dans les enums

```
enum EventType(name: String, start_time: Datetime, duration: Time) is (  
    conference(name = "Tech Conference", start_time = Datetime"2024-03-15T09:00:00Z", duration = Time"01:00:00"),  
    workshop(name = "Workshop", start_time = Datetime"2024-03-18T14:00:00Z", duration = Time"02:00:00"),  
    meeting(name = "Team Meeting", start_time = Datetime"2024-03-20T10:30:00Z", duration = Time"01:30:00")  
)  
  
enum QueryType(name: String, template: Sql, timeout: Integer) is (  
    select(name = "SELECT", template = Sql"SELECT * FROM table WHERE condition = ?", timeout = 10),  
    insert(name = "INSERT", template = Sql"INSERT INTO table (columns) VALUES (values)", timeout = 5),  
    update(name = "UPDATE", template = Sql"UPDATE table SET column = ? WHERE id = ?", timeout = 10)  
)
```

12.13.8 Exemples d'usage dans les records

```
record Event is  
    title : String  
    start_datetime : Datetime = Datetime"2021-01-01T00:00:00Z"  
    end_date : Date = Date"2021-01-01"  
    duration : Time = Time"01:00:00"  
    location : Geography = Geography"POINT(0.0 0.0)"
```

```

    metadata : Json = Json{"type": "default"}
    binary_data : Bytes = Bytes[0, 0, 0, 0]
    created_timestamp : Timestamp = Timestamp"2021-01-01T00:00:00Z"
end

```

```

record DatabaseQuery is
    query_id : String
    sql_statement : Sql = Sql"SELECT 1"
    description : String = "Default query"
    timeout_seconds : Integer = 30
    parameters : List of String = List[]
    result_format : Json = Json{"format": "json"}
end

```

12.13.9 Exemples d'usage dans les aspects

```

aspect TimeTracking is
    start_time : Datetime = Datetime"2021-01-01T00:00:00Z"
    end_time : Datetime = Datetime"2021-01-01T23:59:59Z"
    duration : Time = Time"00:00:00"
    created_timestamp : Timestamp = Timestamp"2021-01-01T00:00:00Z"
    timezone : String = "UTC"
end

```

```

aspect DataProtection is
    encryption_key : Bytes = Bytes""
    access_control : Json = Json{"level": "public", "encryption": false}
    audit_trail : Json = Json{"enabled": true, "retention_days": 365}
    retention_period : Interval = Interval 7 years
    last_accessed : Datetime = Datetime"2021-01-01T00:00:00Z"
end

```

12.13.10 Exemples d'usage dans les entités

```

entity Meeting is
    MeetingId : String
    StartTime : Datetime

```

```

EndTime : Datetime = Datetime"2021-01-01T23:59:59Z"
Duration : Time = Time"01:00:00"
Location : Geography = Geography"POINT(0.0 0.0)"
Attendees : List of String = List []
Metadata : Json = Json{"type": "meeting"}

aspects {
    TimeTracking {
        start_time: Datetime"2024-03-15T09:00:00Z",
        end_time: Datetime"2024-03-15T17:00:00Z",
        duration: Time"08:00:00",
        created_timestamp: Timestamp"2024-01-01T00:00:00Z"
    },
    DataProtection {
        encryption_key: Bytes"",
        access_control: Json{"level": "private", "encryption": true},
        audit_trail: Json{"enabled": true, "retention_days": 2555},
        retention_period: Interval 5 years,
        last_accessed: Datetime"2024-03-15T14:30:25Z"
    }
}
end

```

12.13.11 Conseils d'utilisation

1. **Format des dates** : Utilisez toujours le format ISO 8601 (YYYY-MM-DDTHH:mm:ssZ)
2. **Géographie** : Préférez le format WKT pour les géométries simples, GeoJSON pour les structures complexes
3. **SQL** : Les requêtes peuvent contenir des paramètres avec ? pour la sécurité
4. **Bytes** : Utilisez les tableaux d'octets pour les données binaires, les chaînes hex/base64 pour l'encodage
5. **Intervalles** : Les intervalles temporels supportent les unités : year, month, day, hour, minute, second
6. **JSON** : Utilisez la syntaxe objet {} pour les structures, la syntaxe chaîne "" pour le JSON brut

Cette annexe couvre tous les littéraux supportés par Meshr-lang avec des exemples pratiques et des cas d'usage réels.

